

On k -sum algorithms for $\{-1, 1\}^m$ vectors

Pabasara Athukorala and Steven D. Galbraith

Department of Mathematics, University of Auckland, New Zealand.
s.galbraith@auckland.ac.nz, path468@aucklanduni.ac.nz

Abstract. We study a new variant of the k -sum problem. In the classical formulation, one is given k lists of binary vectors and must find one vector from each list such that their sum is the zero vector. In our variant, vectors are instead chosen from the set $\{-1, 1\}^m$. First, we analyse how the complexity of the original problem changes when one uses ± 1 vectors. We show that this variant achieves better than square-root complexity when using more than two lists, although its overall complexity for $k \geq 4$ is worse than that for the binary case. As an application of our algorithms, we present a combinatorial attack strategy for a recently proposed variant of the Inhomogeneous Short Integer Solution (ISIS) problem known as the randomised one-more ISIS assumption. Our combinatorial attack is more effective than previous combinatorial attacks on this problem.

We further consider a generalised setting of the k -sum problem in which the target sum is not the zero vector, but a given integer vector $Y \in \mathbb{Z}_p^m$ that is known to be a sum of ± 1 vectors. We study the complexity of reconstructing such a decomposition of Y using k lists of candidate vectors. Our analysis shows that, as the number of lists increases, the problem becomes solvable in approximately cube-root time.

Keywords: Generalised birthday problem, k -sum algorithm, ISIS problem

1 Introduction

Wagner’s algorithm [30] is an elegant approach for solving a k -sum extension of the classic birthday problem. Given k lists of n -bit values, the goal is to select one element from each list so that the XOR of the selected k values equals zero. For the case where $k = 2$, this reduces to the well-known birthday problem, which can be solved in square-root time. Wagner extends this result, providing a cube-root time algorithm for $k = 4$ and additional optimisations for $k > 2$. Due to its effectiveness and simplicity, Wagner’s algorithm and its extensions [11, 25, 26], have become a widely used tool in cryptography and related fields [9, 20, 22].

Over the years, Wagner’s algorithm has found different variations. Wagner’s paper [30] considers a subset-sum problem over $\mathbb{Z}$. Shallue [29] and Joux et al. [17] have independently analysed the k -list algorithm over a field $\mathbb{Z}_p$ and [8] defines the “approximate k -list problem” where the authors have relaxed the original problem and aim to find a set of vectors that sum to a vector of small Hamming weight.

In this paper we consider a natural problem that, as far as we are aware, has not previously been studied: Given k lists of vectors in $\{-1, 1\}^m$, to find a sum of elements that is equal to the zero vector. Intuitively, this problem makes sense, because the distribution of entries in a sum of an even number k of such vectors has zero as the most probable value and tends to stay within $\mathcal{O}(\sqrt{k})$ in absolute value. So it is plausible that a subset of k vectors can be found that sum to zero. Indeed, our algorithmic results show it is possible. This problem is inspired by a cryptanalysis attempt on a new computational assumption by Baldimtsi, Cheng, Goyal and Yadav [4] related to the one-more-ISIS problem.

Hanzlik [15] introduced a new variant of blind signatures called non-interactive blind signatures (NIBS). This allows a signer to generate partial signatures to any recipient, such that

only the intended recipient can extract the blinded signature for a random message. Baldimtsi et al. [4] improved this concept further by giving a concrete analysis of the post-quantum security of NIBS. In their work, Baldimtsi et al. follow a recent design of practical round-optimal (interactive) lattice-based blind signatures by Agrawal, Kirshanova, Stehlé and Yadav [1] and try to adopt a similar approach for the non-interactive blind signatures. However, they observe that the computational assumption (One-more ISIS problem) used in [1] is not stable enough to prove the unforgeability of their NIBS scheme and propose a new computational assumption called the randomised one-more ISIS (rOM-ISIS) problem. They claim that the new variant of the ISIS assumption is more secure than the OM-ISIS assumption used by Agrawal et al. based on some preliminary cryptanalysis. The motivation for this paper is to check the security of the new variant rOM-ISIS against a combinatorial attack based on Wagner’s k -list algorithm. Our findings indicate that one can successfully generate additional valid solutions, enabling an attack on the rOM-ISIS assumption. Then we study the scheme given in [4]. Due to the carefully chosen parameters by Baldimtsi et al. [4], the practical impact of this attack is limited, and the scheme remains secure in its current instantiation.

More recently, the rOM-ISIS assumption has been used to prove the security of several other schemes, including a lattice-based Threshold Blind signatures [12] and a Batched Non-Interactive Blind signature scheme [5]. We argue that the rOM-ISIS assumption falls short of its intended security goals and should need further scrutiny before using in cryptographic constructions. We believe our work is useful for the future study of problems of this nature.

1.1 Our contributions

We first give a new version of the generalised birthday problem for $\{-1, 1\}^m$ vectors. Given lists $L_1, \dots, L_k$ containing vectors sampled independently at random from $\{-1, 1\}^m$, we find $\mathbf{x}_i \in L_i$ for all $i = 1, \dots, k$ that add up to the zero vector. We assume k is a power of 2. In Section 3, we show that this problem can be solved with k -lists in better running time than $2^{m/2}$ operations (even reaching $2^{m/2}$ for this problem is non-trivial by meet-in-the-middle methods, as explained in Appendix A). In the basic 4-list case, we were able to obtain a running time of approximately, $2^{0.4144m}$ and the running time gets better when the number of lists increases. However, it is slightly higher than the complexity of the Wagner’s k -list algorithm for binary vectors, which is not unexpected given the additional complexity of the problem. We refer to Table 2 for exact values.

In Section 4, we give an analysis that allows to make more general asymptotic statements. We use the heuristic that the expected list lengths at intermediate steps of the k -tree algorithm are the same as that of the initial list lengths. This is the same as in Wagner’s analysis. This assumption gives a simple and easy to analyse variant of the algorithm.

As an application of these new variants of the generalised birthday problem, we show how to attack the rOM-ISIS assumption introduced in [4]. For the proposed parameters in [4] the blind signature scheme is secure against such a combinatorial attack. However, we suggest that further scrutiny is needed before claiming that the rOM-ISIS assumption is strong and robust and to be used in building cryptographic schemes.

Finally, we look at a special case of the generalised birthday problem when the sum of the vectors adds up to a special vector $Y \in \mathbb{Z}^m$. We assume that $Y = \sum_{i=1}^n \mathbf{y}_i$ where $\mathbf{y}_i \in \{\pm 1\}^m$ and we analyse the complexity to write Y as a sum of k vectors taken from $\mathbf{x}_i \in L_i$ for $i = 1, \dots, k$ and $\mathbf{x}_i \in \{\pm 1\}^m$. We explore how to use a Wagner like algorithm to solve the

above problem. We will show that we can get better complexity than $2^{m/2}$ if we have more than 8 lists and as the number of lists grows bigger, we can even get close to cube-root complexity.

1.2 Acknowledgements

We thank Ron Steinfeld, and Muhammed Esgin for interesting discussions and suggestions on this work during the Post-Quantum Cryptography Workshop. We also thank the anonymous reviewers of an earlier version of the paper for their suggestions and encouragement.

2 Preliminaries

An m -dimensional lattice $\mathcal{L}$ is a discrete additive subgroup of $\mathbb{R}^m$. Given positive integers n, m, p and a matrix $\mathbf{A} \in \mathbb{Z}_p^{n \times m}$, let $\Lambda_p^\perp(\mathbf{A})$ denote the lattice $\{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = 0 \pmod{p}\}$.

Discrete Gaussians For any vector $\mathbf{c} \in \mathbb{R}^m$ and any real $\sigma > 0$, the (spherical) Gaussian function with standard deviation parameter σ and centre parameter $\mathbf{c}$ is defined as

$$\forall \mathbf{x} \in \mathbb{R}^m, \rho_{\sigma, \mathbf{c}}(\mathbf{x}) = \exp\left(\frac{-\|\mathbf{x} - \mathbf{c}\|^2}{2\sigma^2}\right).$$

The Gaussian distribution is $\mathcal{D}_{\sigma, \mathbf{c}} = \rho_{\sigma, \mathbf{c}}(\mathbf{x})/\sigma^m$. When $\mathbf{c} = 0$, omit the subscript $\mathbf{c}$. The expected norm of the vector sampled from $\mathcal{D}_\sigma$ is given by $\mathbb{E}[\|\mathbf{x}\|] = \sigma\sqrt{m}$.

Note that one can equivalently write the Gaussian function as $\rho_\zeta(\mathbf{x}) = \exp(-\pi\|\mathbf{x}\|^2/\zeta^2)$, where $\zeta = \sigma\sqrt{2\pi}$ is a parameter. Now the distribution is denoted by $\mathcal{D}_\zeta$.

For any set $\mathcal{L} \subset \mathbb{R}^m$, define $\rho_\zeta(\mathcal{L}) = \sum_{\mathbf{x} \in \mathcal{L}} \rho_\zeta(\mathbf{x})$. The discrete Gaussian distribution $\mathcal{D}_{\mathcal{L}, \zeta}$ over $\mathcal{L}$ with parameter ζ is defined by the probability distribution function

$$\rho_{\mathcal{L}, \zeta}(\mathbf{x}) = \frac{\rho_\zeta(\mathbf{x})}{\rho_\zeta(\mathcal{L})}$$

for all $\mathbf{x} \in \mathcal{L}$. The following lemma (Lemma 4.4 of [24]) shows that if the parameter ζ of a discrete Gaussian distribution is large enough, then any vector drawn from this distribution will be short with high probability. Here $\eta_\epsilon(\mathcal{L})$ is the smoothing parameter of a lattice $\mathcal{L}$.

Lemma 1. *For any m -dimensional lattice $\mathcal{L}$, and reals $0 < \epsilon < 1$, $\zeta \geq \eta_\epsilon(\mathcal{L})$, we have*

$$\Pr[\|\mathbf{x}\| > \zeta\sqrt{m} : \mathbf{x} \leftarrow \mathcal{D}_{\mathcal{L}, \zeta}] \leq \left(\frac{1+\epsilon}{1-\epsilon}\right) 2^{-m}.$$

2.1 Computational assumptions

We will encounter the following computational assumptions in the later part of the paper, which are versions of the Short Integer Solution (SIS) problem introduced by Ajtai [2]. Its inhomogeneous version, asking to find a short solution to a linear algebra problem $(\bmod p)$ was formalised later in [23].

Definition 1. (*Inhomogeneous-SIS*) Let p, n, m, β be functions of security parameter λ . An instance of the $\text{ISIS}_{p, n, m, \beta}$ problem is a matrix $\mathbf{A} \leftarrow \mathbb{Z}_p^{n \times m}$ and a vector $\mathbf{y} \leftarrow \mathbb{Z}_p^m$, and a solution to the problem is a vector $\mathbf{z} \in \mathbb{Z}^m$ such that $\|\mathbf{z}\|_2 \leq \beta$ and $\mathbf{A}\mathbf{z} = \mathbf{y} \pmod{p}$.

Agrawal et al. [1] introduce a new member to this family to build blind signatures. The assumption states that it is hard to find $\ell + 1$ short preimages from a set of syndromes S , even when given up to ℓ inversions of arbitrary syndromes not in S , for any polynomially bounded ℓ .

Definition 2. (*One-more ISIS*) Let p, n, m, β, ζ be functions of security parameter λ . Consider the following experiment:

1. A challenger uniformly samples a matrix $\mathbf{A} \in \mathbb{Z}_p^{n \times m}$ and sends $\mathbf{A}$ to the adversary $\mathcal{A}$.
2. $\mathcal{A}$ performs two types of queries in any order.
 - Syndrome queries.** $\mathcal{A}$ requests a challenge vector, to which the challenger replies with a uniformly sampled vector $\mathbf{t} \leftarrow \mathbb{Z}_p^n$. Denote the set of received vectors by S .
 - Preimage queries.** $\mathcal{A}$ queries any vector $\mathbf{t}' \in \mathbb{Z}_p^n$. The challenger will return a short vector $\mathbf{y}' \in \mathbb{Z}^m$ such that $\mathbf{A}\mathbf{y}' \equiv \mathbf{t}' \pmod{p}$ and $\|\mathbf{y}'\| \leq \zeta\sqrt{m}$. Let ℓ denote the number of preimage queries.
3. Finally, $\mathcal{A}$ outputs $\ell + 1$ distinct pairs of the form $\{(\mathbf{y}_j, \mathbf{t}_j)\}_{j \in [\ell+1]}$. $\mathcal{A}$ wins if

$$\forall j \in [j + 1], \quad \mathbf{A}\mathbf{y}_j \equiv \mathbf{t}_j \pmod{p}, \quad \|\mathbf{y}_j\| \leq \beta, \quad \text{and } \mathbf{t}_j \in S.$$

The OM-ISIS assumption states that for every PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins is $\text{negl}(\lambda)$.

Agrawal et al. discuss polynomial time attacks on their new assumption in the same paper. Baldimsti et al. [4] modified the above variant to achieve two new goals. First to protect the scheme from the attackers that query the same preimage more than once and to allow for re-randomisation of the target vectors before answering the preimage queries and thus making the assumption more robust.

Definition 3. (*Randomised OM-ISIS*) Let p, n, m, β, ζ be functions of security parameter λ . Consider the following experiment:

1. The challenger uniformly samples two matrices $\mathbf{A}, \mathbf{B} \in \mathbb{Z}_p^{n \times m}$, and sends $\mathbf{A}, \mathbf{B}$ to adversary $\mathcal{A}$.
2. $\mathcal{A}$ adaptively makes queries of the following types to the challenger, in any order:
 - Syndrome queries.** $\mathcal{A}$ requests for a challenge vector, to which the challenger replies with a uniformly sampled vector $\mathbf{t} \leftarrow \mathbb{Z}_p^n$. We denote the set of received vectors by S .
 - Preimage queries.** $\mathcal{A}$ queries a vector $\hat{\mathbf{t}} \in \mathbb{Z}_p^n$, to which the challenger replies with a short vector $\hat{\mathbf{x}} \in \mathbb{Z}_p^m$ and a vector $\hat{\mathbf{y}} \in \{\pm 1\}^m$ such that $\mathbf{A}\hat{\mathbf{x}} + \mathbf{B}\hat{\mathbf{y}} = \hat{\mathbf{t}}$ and $\|\hat{\mathbf{x}}\| \leq \zeta\sqrt{m}$. Let ℓ denote the total number of preimage queries.
3. Finally, $\mathcal{A}$ outputs $\ell + 1$ distinct tuples of the form $\{(\mathbf{x}_j, \mathbf{y}_j, \mathbf{t}_j)\}_{j \in [\ell+1]}$. $\mathcal{A}$ wins if

$$\forall j \in [\ell + 1], \quad \mathbf{A}\mathbf{x}_j + \mathbf{B}\mathbf{y}_j = \mathbf{t}_j, \quad \|\mathbf{x}_j\| \leq \beta, \quad \mathbf{y}_j \in \{\pm 1\}^m, \quad \text{and } \mathbf{t}_j \in S.$$

The rOM-ISIS $_{p,n,m,\beta,\zeta}$ assumption states that for every PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins is $\text{negl}(\lambda)$.

According to the new assumption, the attacker does not get preimages for the arbitrary target vectors $\hat{\mathbf{t}}$ that it selects, but for a publicly re-randomised vector $\mathbf{t}' = \hat{\mathbf{t}} - \mathbf{B}\mathbf{y}$ for a random $\mathbf{y} \in \{\pm 1\}^m$ chosen by the challenger. They claim that the requirement $\mathbf{y}_j$ must be ± 1 vectors prevent the attacks that were implemented on the OM-ISIS assumption.

Both Definition 2 and 3 provide sample vectors with norm bounded by $\zeta\sqrt{m}$, and ask the attacker to produce vectors of norm bounded by β . The larger the gap between these two values, the better our cryptanalysis will work. In other words, our methods primarily show the danger of choosing β too much larger than $\zeta\sqrt{m}$.

Remark 1. There is a subtlety about these definitions that we need to make more precise for our attacks. The pre-image queries are handled using standard lattice trapdoor techniques. This means the vector $\hat{\mathbf{x}}$ is produced by the SamplePre algorithm with parameter ζ . It follows (see Lemma 1) that with overwhelming probability the length of $\|\hat{\mathbf{x}}\|$ is $\leq \zeta\sqrt{m}$. For the analysis of our later attacks we will use the stronger condition that $\hat{\mathbf{x}}$ is sampled from a Gaussian, not just that it satisfies $\|\hat{\mathbf{x}}\| \leq \zeta\sqrt{m}$.

2.2 Existing k -tree algorithms

Camion and Patarin [10] get the credit for the idea of first using tree based algorithms to solve k -sum problems faster than square-root time. Later Wagner [30] extended their work by considering the general case for k -sum problem for arbitrary groups, by giving the asymptotic complexity of the algorithm and by introducing new applications to cryptanalysis. Wagner’s algorithm will be the foundation in our work.

We quote the original k -sum problem:

Definition 4. *Given k lists $L_1, \dots, L_k$ of elements drawn uniformly and independently at random from $\{0, 1\}^n$, find $x_1 \in L_1, \dots, x_k \in L_k$ such that $x_1 \oplus x_2 \oplus \dots \oplus x_k = \mathbf{0}$.*

Wagner’s k -tree (aka k -sum, or k -lists) algorithm is an elegant and simple algorithm for the k -sum problem. Given k lists of n -bit values, the idea is to create new lists consisting of “partial solutions” (for example, finding pairs (x_1, x_2) such that $x_1 \in L_1$ and $x_2 \in L_2$ and $x_1 \oplus x_2$ is zero on some initial ℓ positions). One then combines elements from the new lists to create even newer lists which are closer to the full solution. Eventually, one wants one final lists that contain at least one solution.

For $k = 2$, this is just the extremely well-known birthday problem which can be solved in square-root time $\tilde{O}(2^{m/2})$. Wagner gives a cube-root algorithm for $k = 4$ and some other interesting applications in cryptanalysis.

Wagner’s algorithm works only when one can extend the size of the lists freely, i.e., in the special case where there are sufficiently many solutions to the k -sum problem. Wagner shows that if initial list lengths are greater than $2^{m/(1+\lg k)}$, then a solution exist. So, if lists of size $\mathcal{O}(2^{m/(1+\lg k)})$ are used, the k -sum problem can be solved in $\mathcal{O}(k \cdot 2^{m/(1+\lg k)})$ time and space. We will always assume that we have enough samples to make full use of the k -list algorithm.

3 Wagner’s algorithm for $\{-1, 1\}^m$

We are interested in vectors chosen uniformly and independently at random from $\{-1, 1\}^m$ and our goal is to develop a Wagner-like algorithm for the following problem in $\mathbb{Z}^m$.

Definition 5. *Given k lists $L_1, \dots, L_k$ of vectors drawn uniformly and independently at random from $\{-1, 1\}^m$, find $\mathbf{x}_1 \in L_1, \dots, \mathbf{x}_k \in L_k$ such that $\mathbf{x}_1 + \mathbf{x}_2 + \dots + \mathbf{x}_k = \mathbf{0}$.*

The basic idea is simple: First find pairs $\mathbf{x}_1 \in L_1$ and $\mathbf{x}_2 \in L_2$ such that the first ℓ_1 entries of the vector $\mathbf{x}_1 + \mathbf{x}_2$ are zero. Create a new list of such vectors $\mathbf{x}_1 + \mathbf{x}_2$. These are “partial solutions” in the sense that the problem is solved with respect to the first ℓ_1 entries of the vector. One difficulty is that the remaining entries of the vector are in $\{-2, 0, 2\}$, but the good news is that 0 is still the most likely value in all the remaining positions.

Our approach is heavily influenced by the work of Minder-Sinclair [25]. We follow the same assumption as that of Wagner for initial list length. That is, we assume the initial list length is as large as needed. On the other hand, we vary the number of positions for which the problem is solved at each step, which is a strategy used by Minder and Sinclair. This problem is more complex than previous works, as at each round the non-zero entries in the vectors come from a bigger set, and so the probability of getting zero entries changes. Therefore, we must address these challenges when analysing the new algorithm.

Notations: From this point on, we use the following notations to name the lists in each level. The i^{th} list at the j^{th} level is denoted by $L_i^{(j)}$. In particular, we start with $L_1^{(0)}, L_2^{(0)}, \dots, L_k^{(0)}$ in the initial level. In the next level we have $L_1^{(1)}, L_2^{(1)}, \dots, L_{k/2}^{(1)}$, and so on down to just $L_1^{(q)}$ at the last level where $k = 2^q$. Moreover, $\mathbf{x}_i^{(j)}$ holds implicitly to represent any vector in the corresponding list $L_i^{(j)}$. The addition (+) is the usual component-wise vector addition of $\{-1, 1\}^m$ vectors.

3.1 Overview of algorithm

We assume $k = 2^q$ is a power of 2. Given an instance of the generalised birthday problem as outlined in Definition 5, the k -tree algorithm for $\{-1, 1\}^m$ vectors operates over q levels. At each level, pairs of lists (of partial solutions) are merged to form a new list (of vectors that are closer to a full solution), reducing the total number of lists by half in every round.

Let us re-name the initial k lists as $L_i^{(0)}$. In the first level, the lists $L_1^{(0)}$ and $L_2^{(0)}$ are merged to form a new list $L_1^{(1)}$, the lists $L_3^{(0)}$ and $L_4^{(0)}$ are combined into a list $L_2^{(1)}$, and the process continues for the remaining pairs. Specifically, the new list is composed of all the sums $\mathbf{x}_{2i-1}^{(0)} + \mathbf{x}_{2i}^{(0)}$ with $\mathbf{x}_{2i-1}^{(0)} \in L_{2i-1}^{(0)}$ and $\mathbf{x}_{2i}^{(0)} \in L_{2i}^{(0)}$ for $(i = 1, \dots, k/2)$ such that $\mathbf{x}_{2i-1}^{(0)} + \mathbf{x}_{2i}^{(0)}$ is zero on the first ℓ_1 coordinates. Note that after merging, the new lists contain vectors of the form $\{-2, 0, 2\}^m$. The integer ℓ_1 is a parameter of the algorithm that is to be determined for optimal performance. We say that the first round eliminates ℓ_1 coordinates or equivalently make first ℓ_1 entries zero.

It is important that the merging is done efficiently. Let $L_1^{(0)}$ and $L_2^{(0)}$ be lists to merge, and suppose $|L_1^{(0)}| \leq |L_2^{(0)}|$. We impose the natural lexicographic ordering on vectors in $\mathbb{Z}^m$. Sort $L_1^{(0)}$ with respect to this ordering. This takes $O(|L_1^{(0)}| \log(|L_1^{(0)}|))$ operations. Then, for each element in $L_2^{(0)}$, look up in the sorted list $L_1^{(0)}$. Trying all values and searching takes $O(|L_2^{(0)}| \log(|L_1^{(0)}|))$ operations. So the total cost to merge the lists is bounded by $O(|L_2^{(0)}| \log(|L_1^{(0)}|))$ operations.

Subsequent rounds follow a similar process to the first one. In the second round, new lists $L_1^{(2)}, \dots, L_{k/4}^{(2)}$ are formed from $L_1^{(1)}, \dots, L_{k/2}^{(1)}$, by eliminating the next sequence of ℓ_2 entries. This results in the vectors in the lists $L_1^{(2)}, \dots, L_{k/4}^{(2)}$ having zeros in the first $\ell_1 + \ell_2$ coordinates, while the remaining take values from the set $\{\pm 4, \pm 2, 0\}$.

By iterating this procedure for q rounds, we obtain a final single list containing vectors that are zero on the first $\sum_{j=1}^q \ell_j$ entries. Each of these vectors is a sum of the form $\sum_{i=1}^k \mathbf{x}_i^{(0)}$ with

$\mathbf{x}_i^{(0)} \in L_i^{(0)}$. Since the goal is to find sums that are zero on all m entries, the final list will contain sums of the desired form if the following condition holds

$$\sum_{j=1}^q l_j \geq m. \quad (1)$$

Since the input lists are random, the length of the lists at all levels are random variables. We will write m_j for the expected size of a list at level j . The distribution of m_j is determined by the initial list length m_0 and the values $l_1, \dots, l_j$. Note that, at the last step, m_q represents the total number of solutions found by the algorithm. So our goal is to ensure that

$$m_q = |L_1^{(q)}| \geq 1. \quad (2)$$

The choice of l_j has a significant effect on the behaviour of the algorithm since it determines both the number of entries eliminated and the expected list length at each level. Since the running time is essentially proportional to the sum of the list lengths at each step, achieving optimal performance requires selecting l_j in such a way that the expected list length, $\mathbb{E}[m_j]$, is not too large for all $1 \leq j \leq q - 1$ while satisfying the constraints (1) and (2).

Figure 1 illustrates the basic idea behind the algorithm for the case of $k = 8$ lists. Note that in the figure, we set $l_3 = m - l_1 - l_2$ since we are looking for a vector of dimension m .

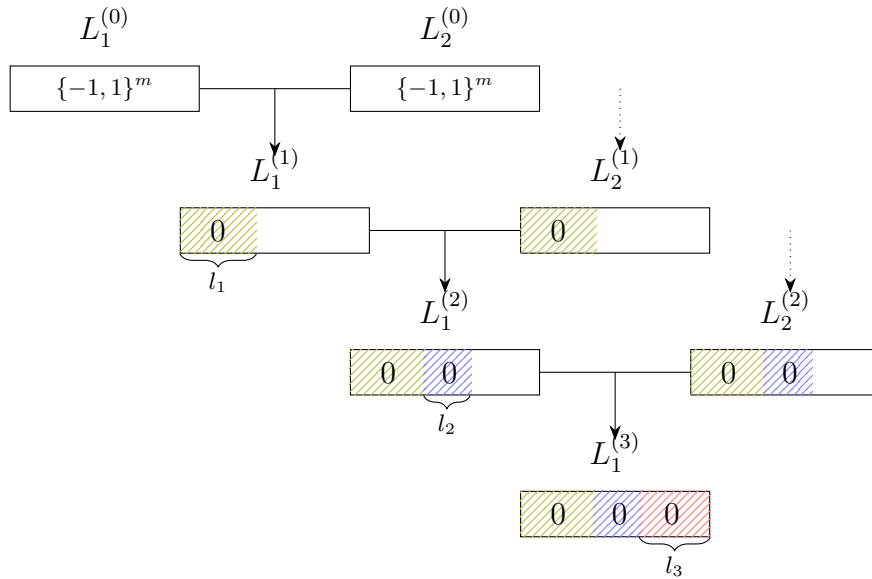


Fig. 1: 8-list algorithm

3.2 Probability Calculation

In this section, we discuss how to compute the relevant probability distributions for integer vectors. Unlike in the binary case, when the k -list algorithm starts with vectors from $\{-1, 1\}^m$, the range of possible vector entries expands at each level of the algorithm. For instance, after

the first merging step, the vectors lie in $\{-2, 0, 2\}^m$, and in the following step they expand further to $\{-4, -2, 0, 2, 4\}^m$.

To analyse the performance of the algorithm, we need to characterise the distribution of the vector entries at each level, as the success probability of zeroing out a subset of coordinates depends directly on this distribution. For example, in the second step, an entry of -2 arises uniquely from summing two -1 values, an entry of 2 from summing two $+1$ values, while a zero can result in two distinct ways: either $(-1) + 1$ or $1 + (-1)$.

The goal of this section is to derive the general distribution of vector entries at each merging step and, in particular, to compute the probability that a given set of coordinates equals zero.

Rademacher Distribution and Random walks. A discrete probability distribution in which a random variable X takes the value $+1$ with probability 0.5 and -1 with probability 0.5 is known as the *Rademacher distribution*. A sum of independent Rademacher variables can be interpreted as a simple symmetric random walk on the integer line, where each step moves either one unit to the left or right with equal probability. In other words, adding random $\{1, -1\}$ values corresponds to a one-dimensional random walk, which is sometimes referred to as a “drunkard’s walk.”

In such a walk, after n steps, the maximum possible position is n and the minimum is $-n$. The distribution of possible sums at each step follows a binomial pattern, which can be derived from the properties of *Pascal’s triangle*. Pascal’s triangle is a triangular array of binomial coefficients and in our situation, it can be considered as probabilities of a random walk in $\mathbb{Z}$ scaled by 2^n , as follows. Assume the random walk starts in the origin (0) , and goes left (-1) or right $(+1)$ with equal probability $1/2$. The number of different walks of n steps where each step is $+1$ or -1 is 2^n . Let X_n denote the random variable representing the position after n steps. From now on we assume n is an even number, since otherwise there is no way to reach zero. The probability of being at position $p' \in \mathbb{Z}$ after n steps with $-n \leq p' \leq n$ is given by

$$\Pr(X_n = p') = \begin{cases} 2^{-n} \binom{n}{\frac{n+p'}{2}} & \text{if } n + p' \text{ is even} \\ 0 & \text{if } n + p' \text{ is odd} \end{cases} \quad (3)$$

Thus, the number of distinct, non-zero probabilities is $n + 1$, corresponding to the positions $p' = -n + 2i$ for $0 \leq i \leq n$. The i^{th} non-zero probability corresponds to $\Pr(X_n = -n + 2i) = 2^{-n} \binom{n}{i}$.

This binomial distribution forms the foundation for understanding how vector entries evolve in our algorithm. Since we are pairing the lists and adding the vectors from these lists at a time, note that after j levels in our algorithm, we have added 2^j many vectors from the original $\{-1, 1\}^m$ input lists. Therefore, at each level j of the k -tree algorithm, we set $n = 2^j$ in the Pascal’s triangle. This is because, at each level, you have taken 2^j steps in a symmetric random walk over $\mathbb{Z}$. To illustrate, at level 1, we compute sums like $\mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)}$ and at level 2, we sum $\mathbf{x}_1^{(1)} + \mathbf{x}_2^{(1)} = \mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)} + \mathbf{x}_3^{(0)} + \mathbf{x}_4^{(0)}$ and so on. Thus, by level j , each resulting vector is the sum of 2^j vectors from the initial lists.

After taking n steps, you need to find the probability of landing on 0. That means you need to calculate $\Pr(X_n = p')$ where, $p' = 0$ which is given by $2^{-n} \binom{n}{n/2}$. However, at each step, we require a block of l_j entries in the vector to be zero. Therefore, we have to raise the above probability to the power l_j to calculate the actual probability ($\Pr_j$) of getting zero in the required l_j coordinates of these vectors at each step $1 \leq j \leq q - 1$. At the end of the algorithm

we need to get a zero vector and hence all the l_j coordinates should add up to the dimension m of the vector. Therefore, in the final step ($j = q$), the probability must be raised to the power $l_q = m - \sum_{j=1}^{q-1} l_j$.

In summary, the probability that a random vector at a given level j has l_j zero entries is given by

$$\Pr_j = \left(2^{-n} \binom{n}{n/2} \right)^{l_j} \quad (4)$$

where $n = 2^j$ and l_j is a parameter to be determined.

3.3 Expected List Length

Once we have the probabilities of getting zero at each step, we can determine the expected list length at these steps in the k -lists algorithm. We define each list $L_i^{(j)}$ as follows. $L_i^{(j)}$ contains pairs of the form $(\mathbf{x}_i^{(j)}, \pi_i^{(j)})$, where $\mathbf{x}_i^{(j)} \in \mathbb{Z}^m$ is a vector constructed at level j , and $\pi_i^{(j)} = (\pi_{2i-1}^{(j-1)} \parallel \pi_{2i}^{(j-1)})$ with $\pi_i^{(0)} = \mathbf{x}_i^{(0)}$, is a back-pointer indicating how $\mathbf{x}_i^{(j)}$ was formed. The symbol $\parallel$ indicates the concatenation of two strings. The initial lists are,

$$L_i^{(0)} = \left\{ (\mathbf{x}_i^{(0)}, \pi_i^{(0)}) \mid \mathbf{x}_i^{(0)} \leftarrow \{-1, 1\}^m \right\},$$

where $\pi_i^{(0)} = \mathbf{x}_i^{(0)}$ is the vector itself, that used to keep track of these initial vectors. Then, for $i = 1, \dots, k/2^j$ at each level $j = 1, \dots, \log k$, we define the list as

$$L_i^{(j)} = \left\{ (\mathbf{x}_i^{(j)}, \pi_i^{(j)}) \mid \begin{array}{l} (\mathbf{x}_{2i-1}^{(j-1)}, -) \in L_{2i-1}^{(j-1)}, \quad (\mathbf{x}_{2i}^{(j-1)}, -) \in L_{2i}^{(j-1)}, \\ \mathbf{x}_i^{(j)} = \mathbf{x}_{2i-1}^{(j-1)} + \mathbf{x}_{2i}^{(j-1)}, \quad (\mathbf{x}_i^{(j)})_{l_1+\dots+l_j} = \mathbf{0} \\ \pi_i^{(j)} = (\pi_{2i-1}^{(j-1)} \parallel \pi_{2i}^{(j-1)}) \end{array} \right\}. \quad (5)$$

In the definition above, we use $(\mathbf{x}_{2i-1}^{(j-1)}, -) \in L_{2i-1}^{(j-1)}$ and $(\mathbf{x}_{2i}^{(j-1)}, -) \in L_{2i}^{(j-1)}$ to indicate that we only take the *first entry* of each pair in the list, i.e., the vector itself, in order to compute the sum. The second entry (the back-pointer) is preserved in the new pair that records how this sum was formed. This construction ensures that each vector $\mathbf{x}_i^{(j)}$ is the sum of two vectors from level $j-1$ that passed the merging condition on the specific coordinates (l_j), and that we retain sufficient information to trace back to the original inputs at level 0.

Remark 2. The notation $(\mathbf{x}_i^{(j)})_{l_1+\dots+l_j} = \mathbf{0}$ means that the first $l_1 + \dots + l_j$ coordinates of $\mathbf{x}_i^{(j)}$ are zero. However, at each level j one only needs to consider the last l_j block of coordinates, since the earlier blocks are already guaranteed to be satisfied. Therefore, when there is no risk of confusion, we sometimes simply write $(\mathbf{x}_i^{(j)})_{l_j} = \mathbf{0}$.

We treat each $\mathbf{x}_i^{(j)}$ as an independent random vector and also, we need these vectors $\mathbf{x}_i^{(j)}$ to satisfy the special property of having zero entries in specific positions. Suppose we are starting with random lists of size m_0 and write m_j for the expected length of a list at the level j of the algorithm. The distribution of m_j is determined by the values m_0 and probability $\Pr_j$ of getting zero at l_j positions.

For example, consider the expected list length at step 1 of the k -tree algorithm. From Equation (4) we have

$$\Pr_1 = [\Pr(X_2 = 0)]^{l_1} = \left(\frac{1}{2}\right)^{l_1}$$

when $\mathbf{x}_1^{(0)}, \mathbf{x}_2^{(0)}$ are chosen uniformly at random such that $\mathbf{x}_1^{(0)} \in L_1^{(0)}$ and $\mathbf{x}_2^{(0)} \in L_2^{(0)}$. We started with lists of size m_0 . Then by the linearity of expectation, the average number of $\mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)}$ in the new list is

$$m_1 = \mathbb{E}[|L_1^{(1)}|] = |L_1^{(0)}| |L_2^{(0)}| \Pr_1 = m_0^2 \left(\frac{1}{2}\right)^{l_1}.$$

Analogously one can compute the expected size of the list $L_2^{(1)}$ of all sums of vectors in $L_3^{(0)}$ and $L_4^{(0)}$ such that first l_1 entries are zero. Similarly, the expected list size of each $L_i^{(1)}$ will be equal to the product of probability $\Pr_1$ and the size of the two merging lists $L_{2i-1}^{(0)}$ and $L_{2i}^{(0)}$ for $i = 1, \dots, k/2$.

We keep iterating this way for q rounds. The expected list length at the j^{th} step is given by $m_j = m_{j-1}^2 \Pr_j$. We can write this as

$$m_j = \prod_{i=1}^j (\Pr_i)^{2^{j-i}} m_0^{2^j}, \quad (6)$$

and at the final step $j = q$,

$$m_q = \mathbb{E}[|L_1^{(q)}|] = \prod_{i=1}^q (\Pr_i)^{2^{q-i}} m_0^{2^q}.$$

After q steps, we want a final list containing at least one vector which is ideally a zero vector or the sum of the first l_j entries ($\sum_{j=1}^q l_j \geq m$) are zero. So our goal is to ensure that $m_q = \mathbb{E}[|L_1^{(q)}|] \geq 1$.

In practice, one usually set up the algorithm so that it outputs a list with several solutions to the problem, for example see Table 4 where the values m_q range between 18 and nearly 5000. Algorithm 1 gives the general formulation of the k -list algorithm. The algorithm returns a set of vectors from the initial k lists that add up to zero or $\perp$ to indicate that none found. If we start with large enough sets, the algorithm succeeds with high probability and returns a solution.

3.4 Running Time and Optimisation Problem

To derive the running time estimate for the k -lists algorithm, we carefully account for the cost of operations at each level of the list-merging process. At each internal step $j \in \{1, \dots, q-1\}$, the algorithm maintains a collection of 2^{q-j} intermediate lists, each containing approximately m_j elements, resulting from sorting and searching elements from the previous level. We perform sorting of the lists according to lexicographic order on vectors in $\mathbb{Z}^m$, which has a cost $\mathcal{O}(m_j \log m_j)$ per list. The cost of searching for matching elements is of the same order, under the assumption that all lists at level j are of equal size m_j . Hence, the total cost to merge list pairs at level j is approximately $2^{q-j} m_j \log m_j$.

Algorithm 1 k -List Algorithm for $\{-1, 1\}^m$ vectors.

Input: $k = 2^q$, $L_i^{(0)}$ for $i = 1, \dots, k$.

Output: $(\mathbf{x}_1^{(0)}, \dots, \mathbf{x}_k^{(0)})$ or $\perp$

```
1: for  $j = 1, \dots, q$  do
2:   Choose  $l_j$  such that  $\sum_{j=1}^q l_j = m$ .
3:   for  $i = 1, \dots, k/2^j$  do
4:      $L_i^{(j)} \leftarrow \emptyset$ 
5:     Sort  $L_{2i-1}^{(j-1)}$  with respect to the first coordinate,  $\mathbf{x}_{2i-1}^{(j-1)}$ 
6:     for each  $(\mathbf{x}_{2i}^{(j-1)}, \pi_{2i}^{(j-1)}) \in L_{2i}^{(j-1)}$  do
7:       Find all  $(\mathbf{x}_{2i-1}^{(j-1)}, \pi_{2i-1}^{(j-1)}) \in L_{2i-1}^{(j-1)}$  such that  $(\mathbf{x}_{2i-1}^{(j-1)} + \mathbf{x}_{2i}^{(j-1)})_{l_1+\dots+l_j} = \mathbf{0}$ .
8:       for all such pairs do
9:          $\mathbf{x}_i^{(j)} \leftarrow \mathbf{x}_{2i-1}^{(j-1)} + \mathbf{x}_{2i}^{(j-1)}$ 
10:         $\pi_i^{(j)} \leftarrow (\pi_{2i-1}^{(j-1)} \parallel \pi_{2i}^{(j-1)})$ 
11:        Add  $(\mathbf{x}_i^{(j)}, \pi_i^{(j)})$  to  $L_i^{(j)}$ 
12:      end for
13:    end for
14:  end for
15: end for
16: if  $|L_1^{(q)}| \geq 1$  then
17:   return  $\pi_1^{(q)}$ 
18: else
19:   return  $\perp$ 
20: end if
```

The total running time is essentially proportional to the sum of the lengths of the lists across all internal steps. Hence, we obtain the total cost for the algorithm:

$$\text{Cost} \approx 2^q m_0 \log(m_0) + 2^{q-1} m_1 \log(m_1) + 2^{q-2} m_2 \log(m_2) + \dots + 2m_{q-1} \log(m_{q-1}). \quad (7)$$

Note that the final list size m_q does not appear in this formula, this is because we do not need to explicitly compute the complete list of all matches, but we can stop as soon as we have found a solution as we are looking for just one solution. For optimal performance of the algorithm, we have to determine the best l_j such that m_j is not too large for any $1 \leq j \leq q-1$. We also have to ensure that the constraint $m_0^k > 2^m$ holds for the existence of a solution to the k -sum problem.

As we discussed earlier, the original k -tree algorithm eliminate a fixed number, $\log m$ bits in each round to keep the list length constant over all rounds, thereby balancing the work done in each round. We follow Minder and Sinclair's method of varying the number of positions solved and thus the intermediate list lengths in each step. First we discuss how to solve this problem as a non-linear optimisation problem and later evaluate the asymptotic behaviour for arbitrary values of m .

In the integer program approach, we adopt a similar framework to that of the Minder and Sinclair [25], but instead of fixing the initial list size m_0 as they did, we allow it to be determined as part of the optimisation process. Specifically, we formulate the problem as a non-linear

optimisation problem to simultaneously determine the optimal values of m_0 and l_j for given values of m and q . This leads to the following integer program, which minimizes the cost (Equation (7)) subject to the given constraints.

$$\begin{aligned}
& \text{minimize Cost} \\
& \text{subject to } \mathbb{E}[|L_1^{(q)}|] \geq 1 \\
& \quad m_j \geq 1 \quad j = 1, \dots, q-1 \\
& \quad l_j \geq 0 \quad j = 1, \dots, q \\
& \quad l_q = m - \sum_{j=1}^{q-1} l_j \\
& \quad m_0 \geq 2^{m/k} \\
& \quad m_j = m_{j-1}^2 \Pr_j \quad j = 1, \dots, q
\end{aligned}$$

The main condition for the algorithm to succeed is that the final list $L_1^{(q)}$ must be non-empty. To ensure this, all intermediate lists must also be non-empty, which motivates the first two constraints. Additionally, we impose the condition that at least one coordinate must be eliminated at each level, and that the total number of eliminated coordinates across all levels must equal m so that the final output is a zero vector of dimension m . The constraint $m_0 \geq 2^{m/k}$ reflects an earlier observation: this lower bound on the initial list size is a necessary condition for the success. Finally, the last constraint defines how the expected sizes of the intermediate lists are computed recursively.

In Table 1 we give an example of the result of solving this optimisation problem for $m = 100$. The cost listed in the right-hand column is a lower bound, since we have not taken into account the sorting/searching and all the low level operations. It is computed as $\text{cost} = 2^q m_0$.

k	l_1	l_2	l_3	l_4	l_5	$\log_2(m_0)$	$\Omega(\text{cost})$
4	44	56				42	2^{44}
8	39	24	38			37	2^{40}
16	35	21	16	27		33	2^{37}
32	32	19	15	12	22	30	2^{35}

Table 1: m_0, l_j and costs for different lists with $m = 100$.

The above analysis gives optimal running times for any specific instance, but it is hard to give general results. Therefore, we perform an asymptotic analysis in the next section.

4 Asymptotic analysis

In this section we make some simplifying assumptions that allow to write down some general statements about the running time. As we discussed earlier, the original k -tree algorithm eliminates a fixed number $\ell = \frac{m}{q+1}$ of bits in each round to keep the list length constant over all rounds, thereby balancing the work done in each round. We use a similar approach to evaluate the asymptotic trend of our algorithm.

To do this we make the simplifying assumption that, as with the original Wagner algorithm, the expected list size is constant throughout all the levels.

Condition 1 *The expected list lengths at the intermediate steps (except for the final list at level q) are equal to the initial list length m_0 .*

From this we calculate the corresponding coordinates to eliminate at each level. When assumed to have equal list lengths, one can find the l_j values (number of positions to solve) at each level in terms of the probability of getting zero in that level (Pr_j) and the number of positions solved in the first step (l_1). So at the end, the initial list length and all the l_j for $j = 2, \dots, q$ values will be in terms of l_1 which we can calculate at the last step in terms of the dimension m .

To extend the above observations, consider an 8-list problem and refer to Figure 1 for clarity. We are given lists $L_1^{(0)}, \dots, L_8^{(0)}$, and our task is to find vectors $\mathbf{x}_i^{(0)} \in L_i^{(0)}$ that add to the zero vector. A necessary condition for a solution to exist is $|L_i^{(0)}| \geq 2^{m/8}$.

In the first level, add elements from $L_1^{(0)}$ and $L_2^{(0)}$ to generate the list $L_1^{(1)}$ such that $(\mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)})_{l_1} = \mathbf{0}$. Similarly, generate the other lists $L_2^{(1)}, \dots, L_4^{(1)}$ by merging the initial lists pairwise. At the second level, the lists $L_1^{(2)}$ and $L_2^{(2)}$ contain vectors whose first $l_1 + l_2$ coordinates are zero. Finally, at level 3, add $\mathbf{x}_1^{(2)} \in L_1^{(2)}$ and $\mathbf{x}_2^{(2)} \in L_2^{(2)}$ to form the final list $L_1^{(3)}$, which contains vectors $\mathbf{x}_1^{(2)} + \mathbf{x}_2^{(2)}$ where the remaining coordinates satisfy $(\mathbf{x}_1^{(2)} + \mathbf{x}_2^{(2)})_{m-l_1-l_2} = \mathbf{0}$.

When we assume that all the intermediate lists have the same list length as the initial lists, the complexity of the algorithm can be analysed as follows. Using the discussion on random walks, we have $\text{Pr}[(\mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)})_{l_1} = \mathbf{0}] = \text{Pr}_1 = (1/2)^{l_1}$ when $\mathbf{x}_1^{(0)}, \mathbf{x}_2^{(0)} \in \{-1, 1\}^m$ chosen independently at random. Then, by the linearity of expectation

$$\mathbb{E}[|L_1^{(1)}|] = |L_1^{(0)}||L_2^{(0)}| \text{Pr}_1 = \frac{m_0^2}{2^{l_1}}.$$

Assuming $\mathbb{E}[|L_1^{(1)}|] \approx m_0$, we simplify to get $m_0 = 2^{l_1}$.

At level 2, we consider vectors from $L_1^{(1)}$ and $L_2^{(1)}$ that sum to zero in the next l_2 coordinates. Using the same reasoning, we write the expected size of $L_1^{(2)}$ as

$$\mathbb{E}[|L_1^{(2)}|] = |L_1^{(1)}||L_2^{(1)}| \text{Pr}_2 = m_0^2 \left(\frac{6}{2^4}\right)^{l_2} \approx m_0.$$

Substituting $m_0 = 2^{l_1}$ gives $l_2 = \frac{-l_1}{\log_2(3/8)} \approx 0.707l_1$. Finally, at the last level, we want the expected size of $L_1^{(3)}$ to be at least 1. That is,

$$\mathbb{E}[|L_1^{(3)}|] = |L_1^{(2)}||L_2^{(2)}| \text{Pr}_3 = m_0^2 \left(\frac{70}{2^8}\right)^{m-l_1-l_2} \geq 1.$$

By substituting the values for m_0 and l_2 derived in the previous steps, we can solve for l_1 in terms of the dimension m of the vectors. That gives the number of entries to eliminate in the first level, and by back substitution we obtain the initial list lengths (m_0) and the number of coordinates to eliminate at the subsequent levels (l_j).

We observed that if set $l_1 = 0.37m$, we can expect to find a solution to the 8-sum problem with non-trivial probability. Then the size of all the list will be approximately $m_0 = 2^{0.37m}$,

and hence the algorithm can be implemented with time and space complexity $\mathcal{O}(2^{0.37m})$, up to logarithmic factors.

This idea can be extended to any $k = 2^q$ number of lists. When k is a power of 2, we have a binary tree of depth $\log_2 k = q$. At each internal node, we maintain the assumption that the expected list length is equal to the initial list length m_0 . This allows us to obtain the m_0 and l_j values in terms of l_1 . More precisely, we have

$$m_0 = 2^{l_1} \quad \text{and} \quad l_j = \frac{-l_1}{\log_2(\text{Pr}_j)} \quad \text{for } j = 2, \dots, q-1, \quad (8)$$

where Pr_j is the probability of landing on zero after $n = 2^j$ steps in a 1-dimensional random walk (i.e., Equation 3 with $p = 0$). In the final level q , we set $l_q = m - \sum_{j=1}^{q-1} l_j$ to ensure that we obtain a zero vector of dimension m . If we want to find one solution for the k -sum problem, set the expected list length to be 1,

$$\mathbb{E}[|L_1^{(q)}|] = m_0^2 (\text{Pr}_q)^{m - \sum_{j=1}^{q-1} l_j} = 1.$$

Substituting the expressions for m_0 and l_j and taking the logarithm, we solve for l_1 as follows;

$$m - \sum_{j=1}^{q-1} l_j = \frac{-2l_1}{\log_2(\text{Pr}_q)}.$$

Since $\sum_{j=1}^{q-1} l_j = l_1 + \sum_{j=2}^{q-1} \frac{-l_1}{\log_2 \text{Pr}_j}$, we obtain

$$l_1 = \frac{m}{1 - \sum_{j=2}^{q-1} \frac{1}{\log_2 \text{Pr}_j} - \frac{2}{\log_2 \text{Pr}_q}}. \quad (9)$$

This framework enables us to solve the k -sum problem for any given dimension m with both time and space complexity bounded by $\mathcal{O}(k \cdot m_0 \log m_0)$. The algorithm operates using intermediate lists of size $\mathcal{O}(m_0)$ at each level of the merging process, where $m_0 = 2^{l_1}$ and l_1 is determined as a function of the dimension m and the collision probabilities Pr_j associated with the merging steps.

Table 2 presents a comparison of how the number of coordinates eliminated at each merging level (denoted by l_j) changes as a function of the number of input lists $k = 2^q$. The values are expressed relative to an arbitrary vector dimension m , that allows us to observe how the elimination changes as the tree depth increases with larger k .

In Figure 2 we illustrate how the running time varies with the number of lists $k = 2^q$ for different dimensions m . A common characteristic of Wagner-type algorithms is a “U”-shaped cost curve: as k increases, the algorithm initially benefits from smaller required list sizes, which reduce the cost of intermediate merging steps. However, these gains are eventually outweighed by the exponential cost introduced by the 2^q factor in the total running time, where $k = 2^q$. This trade-off is particularly evident in the cases of $m = 100$, where the minimum running time is achieved at $k = 2^9$. For larger dimensions ($m = 500, 1024$), the figures do not extend far enough to see the “U” shape, but it indicates that increasing the number of lists beyond $k = 2^{20}$ has little practical advantage.

In Section 6 we follow the same idea to obtain a k -tree like algorithm to determine the complexity to write a given vector Y as a sum of k vectors $\mathbf{x}_i \in \{-1, 1\}^m$ from a given list. First, we apply our methods so far to the rOM-ISIS problem. Note that the rOM-ISIS problem works with vectors of dimension $m = 1024$ and the running time for this algorithm will be too large as shown in Figure 2.

k	l_1	l_2	l_3	l_4	l_5	l_6	l_7	$\log_2(m_0)$
4	$0.4144m$	$0.5856m$						$0.4144m$
8	$0.3603m$	$0.2545m$	$0.3852m$					$0.3603m$
16	$0.3233m$	$0.2285m$	$0.1728m$	$0.2754m$				$0.3233m$
32	$0.2966m$	$0.2096m$	$0.1585m$	$0.1263m$	$0.2090m$			$0.2966m$
64	$0.2762m$	$0.1952m$	$0.1477m$	$0.1176m$	$0.0974m$	$0.1659m$		$0.2762m$
128	$0.2603m$	$0.1839m$	$0.1391m$	$0.1108m$	$0.0917m$	$0.0781m$	$0.1361m$	$0.2603m$

Table 2: Comparison of number of lists ($k = 2^q$), l_j values and initial list lengths m_0 for $j = 1, \dots, q$

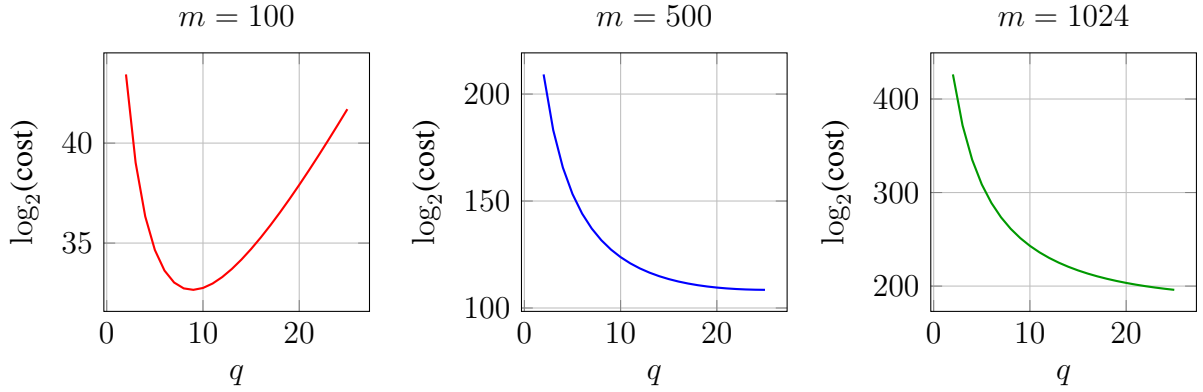


Fig. 2: Variation of cost with the k -list algorithm for $m = 100, 500$, and 1024 and $k = 2^q$.

5 Solving the rOM-ISIS problem

Our work is motivated by a new variant of the ISIS problem introduced by Baldimtsi, Cheng, Goyal and Yadav [4]. They claim that their new variant, called the randomised one-more ISIS (rOM-ISIS) assumption (Definition 3), is more robust than the original one-more ISIS assumption (Definition 2) introduced by Agrawal, Kirshanova, Stehlé and Yadav [1]. In this section we will compare the two problems and give a cryptanalysis strategy for the rOM-ISIS assumption based on the new k -tree algorithms that we introduced.

Over the years, several variants of the ISIS assumption have been proposed and recently, Agrawal et al. [1] introduced a notable new variant called the one-more ISIS (OM-ISIS) assumption to build blind signatures. The assumption states that an adversary cannot efficiently find reasonably short preimages for a set of $\ell + 1$ randomly chosen target vectors, even when given access to a preimage sampling oracle that can generate at most ℓ preimages for adversarially selected targets. The hardness of this problem critically depends on several parameters, notably β , which bounds the norm of the preimage vectors $\mathbf{y}_i$ that the adversary is required to produce, as well as the dimensions m and n of the input matrix $\mathbf{A}$.

The authors have given two preliminary attacks, a combinatorial attack and a lattice attack, for their assumption. All cryptanalysis efforts on OM-ISIS exploit the fact that the attacker can submit any target vector of its choice as a preimage query. Thus, an attacker can potentially request for preimage queries for short vectors, and use those to create an approximate trapdoor. However, the quality of a trapdoor computed this way is much worse than the actual trapdoor,

thus such a trapdoor would be useless if the $\ell + 1$ preimage vectors that the attacker must compute have to be as short as the preimage vectors it received.

In [4] it is suggested that providing an unrestricted preimage query access to the attacker is too strong, and it will make the OM-ISIS susceptible to arbitrary linear combination attacks. To highlight their idea, the following simple attack that finds short preimage vectors without creating a lattice trapdoor is given.

Suppose the attacker makes two preimage queries: one on a zero vector and the other on any non-zero vector $\mathbf{t} \in S$. Let $\mathbf{y}_1$ and $\mathbf{y}_2$ be the respective preimage vectors. That is, $\mathbf{A}\mathbf{y}_1 = \mathbf{0}$ and $\mathbf{A}\mathbf{y}_2 = \mathbf{t}$. Given $\mathbf{y}_1$ and $\mathbf{y}_2$, an attacker can simply output three distinct vectors $\mathbf{y}_2$, $\mathbf{y}_2 - \mathbf{y}_1$, and $\mathbf{y}_2 + \mathbf{y}_1$ as the preimages for the same target vector $\mathbf{t} \in S$.

This would not qualify as an attack on their two-round blind signature scheme, as it requires an adversary to generate preimages on $\ell + 1$ *distinct* challenge vectors. However, the existence of such a simple attack on the computational assumption has raised concern on the security of the system, and therefore Baldimtsi et al. [4] introduce the rOM-ISIS assumption 3 to thwart these attacks.

The new assumption has two main goals; (a)-protect the system from attackers that can ask for preimage queries on the same target vector more than once, (b)-allow for re-randomisation of the target vectors before answering the preimage query phase to make the assumption more robust. To achieve these goals they modify the OM-ISIS assumption against the practical attacks/cryptanalysis efforts given by Agrawal et al. [1]. These attacks on OM-ISIS use the fact that the attacker can learn preimages on arbitrary target vectors of its choice. To this end, Baldimtsi et al. [4] make the challenger “re-randomise” each target vector (independently for each query) before computing its preimage. In turn, this takes away the attacker’s prior advantage from obtaining distinct short preimages for the same target vector (or any target of its choice more generally). For the precise definition of the assumption, refer to Definition 3.

The new assumption says that an attacker does not receive preimages for arbitrary target vectors $\mathbf{t}$ that it selects, but for a publicly re-randomised vector $\mathbf{t}' = \mathbf{t} - \mathbf{B}\mathbf{y}$, where $\mathbf{y}$ is a random ± 1 vector (chosen by the challenger). Baldimtsi et al. [4] claim that the ability to re-randomise the target vector, gives the challenger more flexibility in answering preimage queries and the condition that $\mathbf{y}_j$ must be ± 1 vectors prevents the algebraic attacks that were mounted on the OM-ISIS assumption.

Baldimtsi et al. [4] also discuss some preliminary cryptanalysis against the rOM-ISIS assumption following the attacks suggested by Agrawal et al. [1] on the OM-ISIS assumption. They claim that the new assumption is more robust than the OM-ISIS assumption against the same class of attacks. The authors give a combinatorial attack and a lattice attack as below.

Lattice attack The adversary requests Q preimage queries of zero ($\mathbf{t}_i = \mathbf{0}$) and receives $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i \in [Q]}$ such that $\mathbf{A}\mathbf{x}_j + \mathbf{B}\mathbf{y}_j = \mathbf{0}$, $\mathbf{x}_i$ is short and $\mathbf{y}_i \in \{\pm 1\}^m$. If $\mathcal{A}$ is able to find a collection $a_1, a_2, \dots, a_Q \in \mathbb{Z}_p$ such that $\mathbf{B} \sum_{i \in [Q]} (a_i \mathbf{y}_i) = \mathbf{0}$, then $\alpha := \sum_{i \in [Q]} (a_i \mathbf{x}_i)$ is a vector in the lattice $\Lambda_p^\perp(\mathbf{A})$. If $|a_i| \leq \gamma$ for a given γ for all a_i , then $\mathcal{A}$ can produce a short basis E for $\Lambda_p^\perp(\mathbf{A})$ such that the corresponding Gram-Schmidt basis has norm bounded from above. Now the attacker runs the Babai’s nearest plane algorithm on $(E, \mathbf{u})$ where $\mathbf{u} \in \mathbb{Z}_p^m$ is a solution to rOM-ISIS problem $\mathbf{A}\mathbf{u} + \mathbf{B}\mathbf{y} = \mathbf{t}$. Let $\mathbf{v} \in \Lambda_p^\perp(\mathbf{A})$ be the output. Then $(\mathbf{u} - \mathbf{v}, \mathbf{y})$ is a solution for rOM-ISIS for $\mathbf{t}$.

Combinatorial attack Let $\{\mathbf{y}'_j\}$ be a set of Q vectors in $\{-1, 1\}^m$. The adversary requests preimages for all vectors in the set $A = \{a_i \mathbf{e}_i + \mathbf{B}(\mathbf{y}'_j - n^{-1} \mathbf{1}) \mid \forall i \in [n], \forall j \in [Q], a \in \mathbb{Z}_p\}$, here $\mathbf{e}_i$ is the i -th unit vector and n^{-1} is the modulo inverse $n \pmod{p}$. Note that adding up n vectors of the form $a_i \mathbf{e}_i$ allows constructing any challenge vector $\mathbf{t} \in \mathbb{Z}_p^n$. Suppose $(\mathbf{x}_{i,j}, \mathbf{y}_{i,j})$ are the oracle responses for each query $a_i \mathbf{e}_i + \mathbf{B}(\mathbf{y}'_j - n^{-1} \mathbf{1})$. Then for each $i \in [n]$ suppose we are lucky enough that there is a $j \in Q$ such that $\mathbf{y}'_j = \mathbf{y}_{i,j}$. If $\mathcal{I}$ contains all such pairs (i, j) and J contains the corresponding j , then

$$\mathbf{A} \sum_{(i,j) \in \mathcal{I}} \mathbf{x}_{i,j} + \mathbf{B} \sum_{(i,j) \in \mathcal{I}} \mathbf{y}_{i,j} = \sum_{i \in [n]} a_i \mathbf{e}_i + \mathbf{B} \sum_{j \in J} (\mathbf{y}'_j - n^{-1} \mathbf{1})$$

This simplifies and gives $(\sum_{(i,j) \in \mathcal{I}} \mathbf{x}_{i,j}, \mathbf{1}, \mathbf{t})$ as a solution to rOM-ISIS assumption with $\beta = \zeta \sqrt{nm}$ with non-negligible probability. As the authors note, the probability of finding such a set $\mathcal{I}$ (and J) goes to zero when $Q = o(2^m)$. Hence, this attack is computationally infeasible for any PPT adversary making polynomially many queries. Definitely the attack requires more than $2^{m/2}$ preimage queries and so our attacks are much better.

Apart from these preliminary cryptanalysis attempts, the authors have not done further security analysis and leave it as an open problem.

5.1 A New Algorithm for the rOM-ISIS Problem

We give a new combinatorial attack strategy based on the Wagner's k -tree algorithm. We claim that the security of the rOM-ISIS assumption will depend critically on the size of the parameters selected. Baldimtsi et al. [4] have carried over the Falcon-512 [13] and Agrawal et al. [1]'s choice of parameters for their scheme. In particular, they have set $p = 7349, n = 512$ and $m = 1024$. In this section we observed that the assumption will remain secure for this set of parameters.

The idea behind this strategy is that if an attacker can find a set of $\mathbf{y}$ vectors that sum up to zero using a k -tree algorithm, then the attacker can construct a shortish vector $\mathbf{x}$ that satisfies $\mathbf{A}\mathbf{x} = \mathbf{0}$. Note that the parameter ranges in which one can use the SamplePre algorithm imply the existence of many non-trivial solutions to $\mathbf{A}\mathbf{x} = \mathbf{0}$. Given a solution $(\mathbf{x}_j, \mathbf{y}_j)$ for a given $\mathbf{t}_j$ one gets “one more” solution $(\mathbf{x}_j \pm \mathbf{x}, \mathbf{y}_j)$ for the same $\mathbf{t}_j$. This allows to solve the rOM-ISIS assumption.

At a high level, the attack strategy is as follows. The attacker makes many preimage queries on the zero vector (i.e. $\mathbf{t} = \mathbf{0} \in \mathbb{Z}_p^n$) to get many pairs $(\mathbf{x}_i, \mathbf{y}_i)$ such that $\mathbf{A}\mathbf{x}_i + \mathbf{B}\mathbf{y}_i = \mathbf{0}$, $\mathbf{y}_i \in \{-1, 1\}^m$ and $\|\mathbf{x}_i\| \leq \zeta \sqrt{m}$. Our variant of Wagner's algorithm produces a set $\mathcal{I}$ of size k such that

$$\sum_{i \in \mathcal{I}} \mathbf{y}_i = \mathbf{0}.$$

Then

$$\mathbf{A} \left(\sum_{i \in \mathcal{I}} \mathbf{x}_i \right) = \mathbf{0}.$$

Let $\mathbf{x} = \sum_{i \in \mathcal{I}} \mathbf{x}_i$. With high probability $\mathbf{x} \neq \mathbf{0}$, since each $\mathbf{x}_i$ is produced independently of $\mathbf{y}_i$ using the SamplePre algorithm.

We need to estimate the norm of $\mathbf{x}$. For this we model the samples $\mathbf{x}_i$ as being sampled from a Gaussian with parameter ζ , as discussed in Remark 1. Recall that the Gaussian function

is defined as $\rho(x) = e^{-\frac{\pi\|x\|^2}{\zeta^2}}$ and the expected norm of a vector of dimension m sampled from this distribution is given by $\mathbb{E}[\|x_i\|] = \zeta\sqrt{\frac{m}{2\pi}}$. Since we are adding k such vectors, the expected norm of the resulting vector x is $\|x\| = \zeta\sqrt{\frac{km}{2\pi}}$. For $x_j + x$ to be a valid solution to the rOM-ISIS problem, we require $\|x_j + x\| \leq \beta$, and so $\zeta\sqrt{\frac{(k+1)m}{2\pi}} \leq \beta$.

For simplicity, we assume that the final list contains a single solution to the k -sum problem. If there are many solutions, then the later part of the attack works even more effectively.

We now describe our algorithm for the rOM-ISIS problem in detail. To begin, we select the largest integer k , restricted to powers of two, such that the condition $\zeta\sqrt{\frac{(k+1)m}{2\pi}} \leq \beta$ is satisfied. This choice of k ensures the required norm bound is met for the resulting vector x . We then compute the total number of required samples for the k -tree algorithm as $\tau = km_0$, and proceed by making τ preimage queries on the zero syndrome vector ($t = 0$). These queries yield a list of (x_i, y_i) pairs, and these y_i 's serve as inputs to the k -list algorithm. The algorithm searches for a set of y_i vectors that sum to zero. Suppose the algorithm found the index set $\mathcal{I}$ of size k with $\sum_{i \in \mathcal{I}} y_i = 0$. Upon finding such a set, we can find the corresponding x_i 's and set $x = \sum_{i \in \mathcal{I}} x_i$ such that $Ax = 0$ with the expected norm of x given as $\|x\| \leq \zeta\sqrt{km/(2\pi)}$.

Next, we make an additional $\tau + 1$ syndrome queries to obtain challenge vectors t_j for $j = 1, \dots, \tau + 1$, followed by corresponding preimage queries which yield pairs (x_j, y_j) that satisfy the rOM-ISIS conditions $Ax_j + By_j = t_j$. For each such triple (x_j, y_j, t_j) , we generate an additional solution by computing $x_j \pm x$, and select the sign that minimises the Euclidean norm of the vector. In total, this process outputs $2(\tau + 1)$ valid solution triples of the form (x_j, y_j, t_j) and $(x_j \pm x, y_j, t_j)$. The total number of preimage queries made is $\tau + \tau + 1$ and the number of outputs is $2(\tau + 1)$. So the attack wins as long as all vectors satisfy the required bounds.

The choice of suitable k is to trade off the success probability and the running time. The vector x is a sum of k vectors, x_i and so the expected value of $\|x\|$ is $\zeta\sqrt{km/(2\pi)}$. For the algorithm to succeed we need at least one of $\|x_j \pm x\| \leq \beta$ for every j . The probability this happens depends on how much of a gap there is between $\zeta\sqrt{km}$ and β .

So the tension is between ensuring the success probability is acceptable and also choosing k , according to Figure 2, to lower the running time. In practice, we will choose the parameters so that there are several solutions x coming from the final list, to increase the chances that, for a given x_j , one of our x vectors is such that $\|x_j \pm x\| \leq \beta$.

5.2 Impact on the NIBS scheme

The algorithm in Section 5.1 solves the rOM-ISIS problem, but readers may be concerned about the fact that half the target syndromes are re-used. Indeed, out of the $\ell + 1$ solutions, the first solution we obtained (x_j, y_j, t_j) and the additional solution generated at the end (x, y, t) are distinct tuples but with the same $y = y_j$ and $t = t_j$. It is natural to ask what is the effect of having two tuples with the same t, y on the security of the blind signature scheme?

To answer this question we have to look at the construction of the scheme and how they have used the computational assumption in the security proof. To maintain consistency in notations, we will use different symbols than those in the original paper [4].

In the design of the new non-interactive blind signature (NIBS) scheme, the signer and receiver operate over public matrices $A, B, M \in \mathbb{Z}_p^{n \times 2m}$. To generate a presignature, the signer

first samples a random vector $\mathbf{y} \in \{\pm 1\}^{2m}$ and computes a preimage for the modified syndrome vector $(\mathbf{pk}_R - \mathbf{B}\mathbf{y})$, where $\mathbf{pk}_R$ denotes the receiver's public key. An important observation at this point is the increase in vector dimension to $2m$. The receiver's public key $\mathbf{pk}_R$ is defined as $\mathbf{t} = \mathbf{M}\mathbf{z} + \mathbf{H}(\delta)$, where $\delta \leftarrow \{0, 1\}^\lambda$ and $\mathbf{z} \leftarrow \mathcal{D}_{\mathbb{Z}_q^{2m}, \zeta/m}$ are receiver's secret keys. So the signer samples a short vector $\mathbf{x}$ such that $\mathbf{A}\mathbf{x} = \mathbf{pk}_R - \mathbf{B}\mathbf{y}$.

Given the presignature and nonce pair $(\mathbf{x}, \mathbf{y})$ the receiver creates a non-interactive zero-knowledge (NIZK) proof π attesting to knowledge of valid witnesses. The proof asserts that, given the inputs $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{w}, \delta$, there exist vectors $\mathbf{x}, \mathbf{y}$, and $\mathbf{z}$ satisfying the following conditions:

$$\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} = \mathbf{M}\mathbf{z} + \mathbf{H}(\delta) \quad \wedge \quad \mathbf{w} = \mathbf{M} \begin{bmatrix} \mathbf{z}_\perp \\ \mathbf{x}_\perp \end{bmatrix},$$

$$\mathbf{y} = [y_1, y_2, \dots, y_{2m}]^T, \quad \text{with } y_i \in \{\pm 1\} \text{ for all } i, \quad \|\mathbf{z}\| \leq \frac{\sqrt{2}\beta}{m}, \quad \|\mathbf{x}\| \leq \sqrt{2}\beta.$$

The $\sqrt{2}$ factor in the norm bound arises from the increase in vector dimension to $2m$. Finally, the receiver sets $\mu := (\mathbf{w}, \delta)$ as the message and $\sigma := (\pi, \mathbf{c})$ as the corresponding signature where $\mathbf{c}$ is an encryption of $\mathbf{x} \parallel \mathbf{y} \parallel \mathbf{z}$ using some randomness $r \leftarrow \{0, 1\}^\lambda$. To verify a signature, one simply runs the verification algorithm for the NIZK. For the full construction of the NIBS scheme, refer Section 7 of [4].

Baldimtsi et al. use the rOM-ISIS assumption to prove the one-more unforgeability of their protocol. To win the one-more unforgeability game, the adversary must output $\ell + 1$ message and signature pairs $(\mu_i, \sigma_i)_{i \in [\ell+1]}$ such that $\mu_i \neq \mu_j$ for $1 \leq i < j \leq \ell + 1$ and on input the verification key $\mathbf{vk} = \mathbf{A}, \mu_i$, and σ_i , the verification algorithm accepts and outputs 1 for all $i \in [\ell + 1]$. The message is $\mu = (\mathbf{w}, \delta)$ where $\mathbf{w}$ and δ are defined as above, and the signature is $\sigma = (\pi, \mathbf{c})$.

Suppose an adversary who follows the above attack strategy outputs two solutions with the same $\mathbf{t}$ and $\mathbf{y}$ for the rOM-ISIS assumption (say $\mathbf{t}_i = \mathbf{t}_j$, and $\mathbf{y}_i = \mathbf{y}_j$). Then according to the above definition of $\mathbf{t}$, this means $\delta_i = \delta_j$ and $\mathbf{z}_i = \mathbf{z}_j$. However, since we have different presignatures $(\mathbf{x}_i \neq \mathbf{x}_j)$, this implies that $\mathbf{w}_i \neq \mathbf{w}_j$ and therefore, $\mu_i \neq \mu_j$. Moreover, having same $\mathbf{t}$ or $\mathbf{y}$ does not affect the validity of an instance $(\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{pke.pk}, \mathbf{c}_i, \mathbf{w}_i, \delta_i)$ and therefore the verification algorithm accepts and output 1 for all $i \in [\ell + 1]$. Therefore, we conclude that it is allowed to have the same $\mathbf{t}$ and $\mathbf{y}$ in the $\ell + 1$ distinct solutions and the adversary can win the game.

The NIBS construction provides both the value of β and the norm bound that the short vector $\mathbf{z}$ should satisfy in this scheme. According to this construction $\beta = \zeta\sqrt{m}$ and $\|\mathbf{x}\| \leq \sqrt{2}\beta$. For our resulting vector $\mathbf{x}$, which has dimension $2m$, the expected norm is given by $\zeta\sqrt{\frac{2m(k+1)}{2\pi}}$. This expected norm must satisfy the given norm bound. Therefore, we can determine the number of lists k that can be used while satisfying this condition. To ensure that the norm bound is met, the number of lists should be small with $k \leq 2\pi - 1 \approx 5$. Since we consider k to be powers of 2, this sets $k = 4$. With $m = 1024$, solving a 4-list algorithm will be infeasible with a cost $\mathcal{O}(2^{426})$ as shown in Figure 2 and the running time increases further when the vectors are of length $2m$. Hence, we conclude that the NIBS scheme remains secure for the particular parameter set chosen in [4].

5.3 A way to choose different $\mathbf{t}$'s

We have given an algorithm that solves the rOM-ISIS problem, and have explained its relevance to cryptanalysis of [4]. But it is also a natural question to find an algorithm that solves rOM-ISIS under the additional requirement that all the vectors $\mathbf{t}$ in the output are distinct. We stress that this is not required in the rOM-ISIS problem as stated, nor in the scheme. But it is a natural question to ask and we were able to make some progress on it.

The method we are proposing is similar to the combinatorial attack strategy given in [4] with few tweaks that allow us to use a new k -tree algorithm that we discuss in the next section. The attack is performed in two phases. We give the steps for the first phase of the attack as follows.

Phase I: The attacker first makes $\tau + 1$ syndrome queries, where τ is a parameter to be specified later. Let S denote the set of received challenge vectors:

$$S = \{\mathbf{t}' : \mathbf{t}' \leftarrow \mathbb{Z}_p^n\}, \quad |S| = \tau + 1.$$

Now, construct the set $T = \{a_i \mathbf{e}_i \mid \forall i \in [n], a_i \in \mathbb{Z}_p\}$, where $\mathbf{e}_i$ are the canonical basis vectors of $\mathbb{Z}_p^n$. This set consists of all scalar multiples of the standard basis vectors, resulting in a total size of np . The attacker then proceeds to query the preimage oracle for every element $a_i \mathbf{e}_i \in T$. Since T contains np elements, the total number of queries made to the oracle is exactly np . For each query, the oracle returns a pair $(\mathbf{x}_{i,a_i}, \mathbf{y}_{i,a_i})$ such that the equation $\mathbf{A}\mathbf{x}_{i,a_i} + \mathbf{B}\mathbf{y}_{i,a_i} = a_i \mathbf{e}_i$ holds. Moreover, the vector $\mathbf{x}_{i,a_i}$ is bounded in norm by $\|\mathbf{x}_{i,a_i}\| \leq \zeta\sqrt{m}$, and the vector $\mathbf{y}_{i,a_i}$ lies in the set $\{\pm 1\}^m$.

Note that any vector $\mathbf{t}' \in \mathbb{Z}_p^n$ can be uniquely expressed as a linear combination of basis vectors, namely, $\mathbf{t}' = \sum_{i=1}^n a_i \mathbf{e}_i$ for some coefficients $a_i \in \mathbb{Z}_p$. In other words, any target vector $\mathbf{t}' \in S$ can be written with at most n entries from T . Therefore, the attacker can obtain the following.

$$\mathbf{A} \sum_{i=1}^n \mathbf{x}_{i,a_i} + \mathbf{B} \sum_{i=1}^n \mathbf{y}_{i,a_i} = \sum_{i=1}^n a_i \mathbf{e}_i = \mathbf{t}'. \quad (10)$$

Defining $X = \sum_{i=1}^n \mathbf{x}_{i,a_i}$ and $Y = \sum_{i=1}^n \mathbf{y}_{i,a_i}$, the attacker gets $\mathbf{A}X + \mathbf{B}Y = \mathbf{t}'$ where X, Y are integer vectors. We stress that $Y \notin \{-1, 1\}^m$ and therefore is not a valid solution to the rOM-ISIS assumption. In this phase we found a way to write any $\mathbf{t}' \in S$ in the form $\mathbf{A}X + \mathbf{B}Y$. However, Y is still not in the right form and this leads to the next phase.

Phase II: Now the attacker query the preimage oracle for zero vector many times (say N times) and suppose $(\mathbf{x}_j, \mathbf{y}_j)$ are the oracle responses for each j that satisfies $\mathbf{A}\mathbf{x}_j + \mathbf{B}\mathbf{y}_j = \mathbf{0}$ such that $\|\mathbf{x}_j\| \leq \zeta\sqrt{m}$ and $\mathbf{y}_j \in \{\pm 1\}^m$. If the attacker finds a set $\mathcal{J}$ of $\mathbf{y}$'s that add up to Y , where Y is the sum of $\mathbf{y}_i$'s obtained in phase I, then

$$Y = \sum_{j \in \mathcal{J}} \mathbf{y}_j \quad \text{and} \quad \mathbf{A} \sum_{j \in \mathcal{J}} \mathbf{x}_j + \mathbf{B} \sum_{j \in \mathcal{J}} \mathbf{y}_j = \mathbf{0}$$

simplifies to

$$\mathbf{B}Y = -\mathbf{A} \sum_{j \in \mathcal{J}} \mathbf{x}_j.$$

Putting everything together from phase I and phase II gives

$$\mathbf{t}' = \mathbf{A}X + \mathbf{B}Y = \mathbf{A}X - \mathbf{A} \sum_{j \in \mathcal{J}} \mathbf{x}_j = \mathbf{A}(X - \sum_{j \in \mathcal{J}} \mathbf{x}_j). \quad (11)$$

To produce a complete solution to the rOM-ISIS problem, we choose an unused preimage response for $\mathbf{t} = \mathbf{0}$ out of N queries made. Suppose this response was the pair $(\mathbf{x}_1, \mathbf{y}_1)$. Adding these responses to Equation 11, we have

$$\mathbf{t}' = \mathbf{A}(\mathbf{x}_1 + X - \sum_{j \in \mathcal{J}} \mathbf{x}_j) + \mathbf{B}\mathbf{y}_1.$$

Define

$$\mathbf{x}' := \mathbf{x}_1 + X - \sum_{j \in \mathcal{J}} \mathbf{x}_j \quad \text{and} \quad \mathbf{y}' := \mathbf{y}_1.$$

Then, we have

$$\mathbf{t}' = \mathbf{A}\mathbf{x}' + \mathbf{B}\mathbf{y}',$$

which satisfies the structure of a rOM-ISIS solution.

Up to this stage, we have made $\tau = np + N$ preimage queries to obtain a triple $(\mathbf{x}', \mathbf{y}', \mathbf{t}')$ with the following properties; $\mathbf{x}'$ is an integer vector (a sum of $n + 1 + |\mathcal{J}|$ short vectors, each of norm at most $\zeta\sqrt{m}$), $\mathbf{y}' \in \{\pm 1\}^m$, and $\mathbf{t}' \in S$. Any challenge vector $\mathbf{t}' \in S$ can be written in the required form, $\mathbf{t}' = \mathbf{A}\mathbf{x}' + \mathbf{B}\mathbf{y}'$ using the pool of τ preimage vectors obtained during Phase I and Phase II. In particular, for $|S| = \tau + 1$, every $\mathbf{t}' \in S$ can be expressed as $\mathbf{t}' = \mathbf{A}\mathbf{x}' + \mathbf{B}\mathbf{y}'$, with $\mathbf{x}' \in \mathbb{Z}_p^m$ and $\mathbf{y}' \in \{-1, 1\}^m$ and therefore, the attacker can generate $\tau + 1$ valid rOM-ISIS solutions.

To analyse the quality of these solutions, especially in terms of the norm of $\mathbf{x}'$, recall that each $\mathbf{x}_j$ is sampled from a discrete Gaussian distribution with standard deviation ζ . The expected Euclidean norm of such a vector is approximately $\zeta\sqrt{m/(2\pi)}$. Since $\mathbf{x}'$ is formed by adding together $(n + 1 + |\mathcal{J}|)$ such independent vectors, the expected norm scales linearly with the number of summands. Thus, the expected norm of $\mathbf{x}'$ is approximately:

$$\mathbb{E}[\|\mathbf{x}'\|] = \zeta\sqrt{(n + 1 + |\mathcal{J}|)\frac{m}{2\pi}}.$$

In order for $(\mathbf{x}', \mathbf{y}', \mathbf{t}')$ to be accepted as a valid solution to the rOM-ISIS instance, it is required that $\|\mathbf{x}'\| \leq \beta$, where β is the bound set by the problem definition. This concludes the attack on the rOM-ISIS problem with distinct challenge vectors $\mathbf{t}$'s.

Remark 3. The core computational step in Phase II is to find a subset $\mathcal{J}$ such that $\sum_{j \in \mathcal{J}} \mathbf{y}_j = Y$. This defines a new variant of the classic k -sum problem: given a target vector $Y \in \mathbb{Z}_p^m$, which is itself a sum of n $\{\pm 1\}^m$ vectors, determine whether it can be reconstructed as the sum of other $\{\pm 1\}^m$ vectors. In the next section, we formally introduce and analyse this novel variant.

In the rOM-ISIS attack strategy, the target vector Y is expressed as a sum of at most n vectors, and the work of [4] sets $n = 512$. Therefore, recovering Y can be modelled as a generalised k -sum problem with $k = 2^9$ lists. Under this setting, the cost of finding a matching

combination is approximately $\mathcal{O}(2^{0.31m})$ up to logarithmic factors (See Section 6 for further details).

It is important to note that this cost only accounts for the second phase of the attack. The overall complexity also depends on the cost of Phase I, which involves making preimage queries for vectors in the set T , requiring approximately $\mathcal{O}(np)$ queries. Thus, the total computational complexity of the complete attack strategy is given by $\mathcal{O}(\max(km_0, np))$ where m_0 is the initial list length for the k -sum problem.

6 k -sum algorithm for a given vector

In this section we discuss a variant of the generalised birthday problem motivated by the discussion in Section 5.3. To get a very high level idea on the problem, suppose you are given a vector Y which is a sum of k vectors $\mathbf{y}_i \in \{-1, 1\}^m$. You need to write this vector Y as a sum of k vectors $\mathbf{x}_i \in \{-1, 1\}^m$ from a different given list. You know the vectors $\mathbf{x}_i$, Y and $\mathbf{y}_i$ are selected uniformly and independently at random. We propose a k -list algorithm for this problem. One major difference is that for this problem we are not free to choose the number of lists, instead, we essentially must take fixed k lists.

Definition 6. *Given k vectors $\mathbf{y}_i \in \{-1, 1\}^m$, let $Y = \sum_{i=1}^k \mathbf{y}_i$. Given k lists $L_1, \dots, L_k$ of vectors drawn uniformly and independently at random from $\{-1, 1\}^m$, find $\mathbf{x}_1 \in L_1, \dots, \mathbf{x}_k \in L_k$ such that $\mathbf{x}_1 + \mathbf{x}_2 + \dots + \mathbf{x}_k = Y$.*

In Appendix B we give an analysis to highlights that the fixed-target variant of the problem is significantly more difficult than both the binary case (where $\mathbf{x}_i \in \{0, 1\}^m$) and the zero-sum case involving ± 1 vectors. The k -tree algorithm we design here will broadly follow the same structure as the algorithm previously described for the k -tree problem in Section 4. The primary difference lies in how we handle the partial sums at each level of the algorithm. We use the fact that the target vector Y is itself the sum of random ± 1 vectors, and therefore instead of merging vectors from the lists with the goal of reaching the zero vector, we now merge them to match specific partial sums of the vectors $\mathbf{y}_j$.

The following notation holds implicitly throughout the chapter. We assume $k = 2^q$. $Y = \sum_{i=1}^k \mathbf{y}_i$ where $\mathbf{y}_i \in \{-1, 1\}^m$ are chosen uniformly and independently at random. Similar to the previous discussions, $\mathbf{x}_i^{(j)} \in L_i^{(j)}$ represent any vector $\mathbf{x}$ from the i^{th} list at the j^{th} level of the k -tree where $i = 1, \dots, k/2^j$ and $j = 1, \dots, q$. The symbol $\parallel$ denotes the concatenation of vectors, e.g., $\pi_1^{(1)} = (\pi_1^{(0)} \parallel \pi_2^{(0)}) = (\mathbf{x}_1^{(0)}, \mathbf{x}_2^{(0)})$.

6.1 Problem formulation

We now illustrate the analysis using an instance of the k -list problem with $k = 8$ and refer the readers to Figure 3 for a visual overview of the algorithmic structure. To begin the k -list algorithm, we are given eight initial lists $L_1^{(0)}, \dots, L_8^{(0)}$, each of size m_0 .

$$L_i^{(0)} = \left\{ (\mathbf{x}_i^{(0)}, \pi_i^{(0)}) \mid \mathbf{x}_i^{(0)} \leftarrow \{-1, 1\}^m \right\}.$$

We define, $\pi_i^{(0)} = \mathbf{x}_i^{(0)}$. The task is to find vectors $\mathbf{x}_1^{(0)}, \dots, \mathbf{x}_8^{(0)}$ such that their sum equals the target vector, $Y = \sum_{i=1}^8 \mathbf{y}_i$, where each $\mathbf{y}_i$ is a random vector with entries in $\{\pm 1\}$.

Level 1. At the first level, we merge the lists pairwise. For example, to construct $L_1^{(1)}$, we consider all pairs of vectors from $L_1^{(0)} \times L_2^{(0)}$ such that their sum matches the first l_1 coordinates of the corresponding partial sum $\mathbf{y}_1 + \mathbf{y}_2$. We denote this matching condition as $(\mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)})_{l_1} = (\mathbf{y}_1 + \mathbf{y}_2)_{l_1}$.

$$L_1^{(1)} = \left\{ \left(\mathbf{x}_1^{(1)}, \pi_1^{(1)} \right) \left| \begin{array}{l} (\mathbf{x}_1^{(0)}, -) \in L_1^{(0)}, \quad (\mathbf{x}_2^{(0)}, -) \in L_2^{(0)}, \\ \mathbf{x}_1^{(1)} = \mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)}, \quad (\mathbf{x}_1^{(1)})_{l_1} = (\mathbf{y}_1 + \mathbf{y}_2)_{l_1} \\ \pi_1^{(1)} = (\pi_1^{(0)} \parallel \pi_2^{(0)}) \end{array} \right. \right\}.$$

In the definition above, we use $(\mathbf{x}_1^{(0)}, -) \in L_1^{(0)}$ and $(\mathbf{x}_2^{(0)}, -) \in L_2^{(0)}$ to indicate that we only take the *first entry* of each pair in the list, i.e., the vector itself, in order to compute the sum. The back-pointer, $\pi_i^{(j)}$ is preserved in the new pair that records how this sum was formed.

Similarly, we generate the lists $L_2^{(1)}, L_3^{(1)}, L_4^{(1)}$ by merging $L_3^{(0)}$ with $L_4^{(0)}$, $L_5^{(0)}$ with $L_6^{(0)}$, and $L_7^{(0)}$ with $L_8^{(0)}$, respectively.

Level 2. At the second level, we merge the lists from the previous level. For instance, to construct $L_1^{(2)}$, we consider pairs from $L_1^{(1)} \times L_2^{(1)}$ such that the sum of the corresponding vectors matches the next l_2 coordinates of $\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{y}_3 + \mathbf{y}_4$. Recall that the first entry of the elements of $L_1^{(1)}$ are of the form $\mathbf{x}_1^{(1)} = \mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)}$, and that of $L_2^{(1)}$ are of the form $\mathbf{x}_2^{(1)} = \mathbf{x}_3^{(0)} + \mathbf{x}_4^{(0)}$, so their sum yields $\mathbf{x}_1^{(1)} + \mathbf{x}_2^{(1)} = \mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)} + \mathbf{x}_3^{(0)} + \mathbf{x}_4^{(0)}$. The corresponding matching condition is,

$$(\mathbf{x}_1^{(1)} + \mathbf{x}_2^{(1)})_{l_1+l_2} = (\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{y}_3 + \mathbf{y}_4)_{l_1+l_2}.$$

Note that one only needs to check the last l_2 coordinates, since the first l_1 are already guaranteed to be correct from the previous step. Similarly, we generate $L_2^{(2)}$ from $L_3^{(1)} \times L_4^{(1)}$, matching against the l_2 coordinates of $\mathbf{y}_5 + \mathbf{y}_6 + \mathbf{y}_7 + \mathbf{y}_8$.

By construction, the list $L_1^{(2)}$ contains vectors whose first $l_1 + l_2$ coordinates match those of $\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{y}_3 + \mathbf{y}_4$, and $L_2^{(2)}$ matches those of $\mathbf{y}_5 + \mathbf{y}_6 + \mathbf{y}_7 + \mathbf{y}_8$.

Level 3. Finally, in the third level, we add the first entries of $(\mathbf{x}_1^{(2)}, \pi_1^{(2)}) \in L_1^{(2)}$ and $(\mathbf{x}_2^{(2)}, \pi_2^{(2)}) \in L_2^{(2)}$ to generate the final list containing vectors $\mathbf{x}_1^{(3)} = \mathbf{x}_1^{(2)} + \mathbf{x}_2^{(2)}$, where $(\mathbf{x}_1^{(2)} + \mathbf{x}_2^{(2)})_{m-l_1-l_2} = (\sum_{i=1}^8 \mathbf{y}_i)_{m-l_1-l_2}$. Since the vectors in $L_1^{(2)}$ and $L_2^{(2)}$ already satisfy the first $l_1 + l_2$ coordinates of the target Y , it suffices to check that their sum matches the remaining $m - l_1 - l_2$ coordinates. Each element in the final list, $L_1^{(3)}$ is of the form $(\mathbf{x}_1^{(3)}, \pi_1^{(3)})$ where $\pi_1^{(3)}$ has a chain of back pointers to the initial vectors $\mathbf{x}_i^{(0)} \in L_i^{(0)}$, ensuring that $\mathbf{x}_1^{(3)} = \sum_{i=1}^8 \mathbf{x}_i^{(0)}$. Thus, the final list contains vectors that sum to the target vector Y , as desired.

Apart from the fact we merge to sums of the $\mathbf{y}_j$, the details are the same as the previous k -list algorithm. The length of the intermediate lists are variable, but we can model the algorithm using the expected values of the lengths of the lists.

We make the assumption that the expected lists lengths are equal to that of the initial list length m_0 , as in Section 4. Later in Remark 4 we will give experimental evidence to support this assumption. The goal is to ensure that the final list $L_1^{(q)}$ is non-empty. So we need $\mathbb{E}[|L_1^{(q)}|] \geq 1$. The algorithm will determine the optimal l_j for $j = 1, \dots, q$ and m_0 for the k -sum problem to have a solution. Figure 3 gives the main idea behind the new k -list algorithm for $k = 8$.

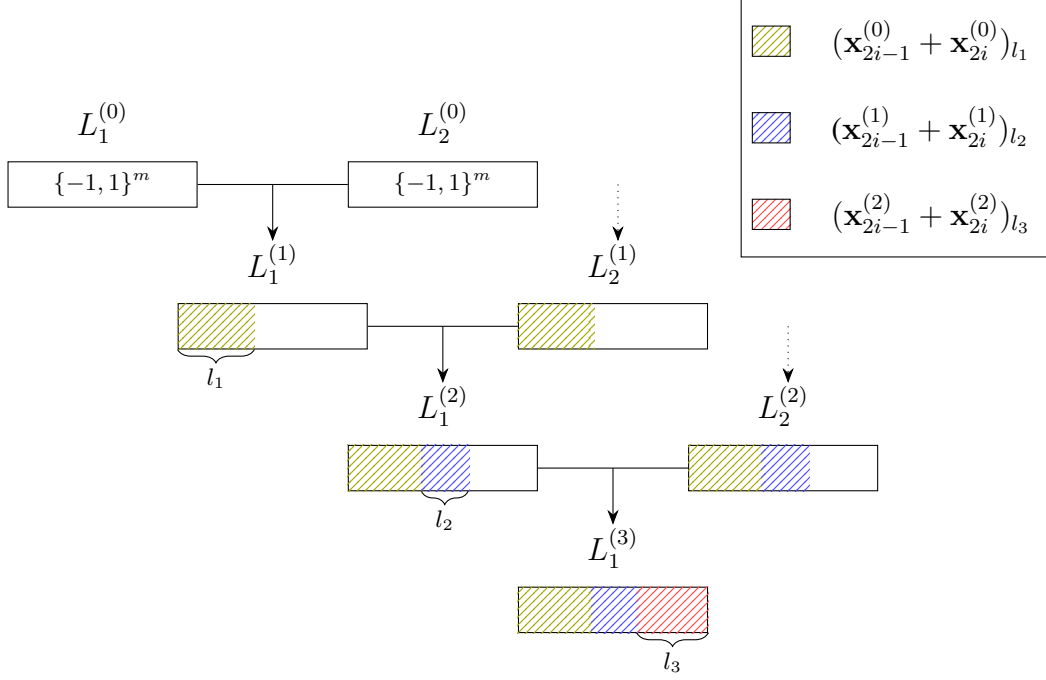


Fig. 3: 8-list algorithm to find a set of vectors that sum to a given vector $Y = \sum_{i=1}^8 \mathbf{y}_i$

6.2 Probability Calculation for a fixed Y

The main challenge in analysing this algorithm lies in computing the probability that a sum of vectors matches a corresponding sum of the $\mathbf{y}_i$. As discussed in Section 3.2, we again use the insights from random walks and Pascal's triangle to estimate the success probabilities.

Recall that each vector $\mathbf{y}_i \in \{-1, 1\}^m$ is sampled independently and uniformly at random, and the target vector $Y = \sum_{i=1}^k \mathbf{y}_i$ is fixed and given to the algorithm. The distribution of Y and intermediate sums such as $\mathbf{y}_1 + \mathbf{y}_2$, or $\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{y}_3 + \mathbf{y}_4$, follows the distribution given by Equation (3).

Level 1. At level 1 of the algorithm, our goal is to match the first l_1 coordinates of the sum $\mathbf{y}_1 + \mathbf{y}_2$. Each coordinate in this sum takes a value in the set $\{-2, 0, 2\}$. Assuming $\mathbf{y}_1$ and $\mathbf{y}_2$ are uniformly random and independent vectors with entries in $\{-1, 1\}$, the distribution of their sum resembles a simple two-step random walk. For the analysis of a fixed Y , we take the following example Y with the expected distribution of values across the first l_1 coordinates as $l_1/4$ entries equal to $+2$, $l_1/4$ entries equal to -2 , and $l_1/2$ entries equal to 0 .

The lists $L_1^{(0)}$ and $L_2^{(0)}$ contain vectors $\mathbf{x}_1^{(0)}$ and $\mathbf{x}_2^{(0)}$, respectively, which are independent random vectors with entries in $\{-1, 1\}$. Let $\mathbf{x}_1^{(1)} = \mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)}$. We are interested in the probability that the first l_1 coordinates of $\mathbf{x}_1^{(1)}$ match those of $\mathbf{y}_1 + \mathbf{y}_2$.

Since $\mathbf{x}_1^{(1)}$ is a sum of independent random ± 1 vectors, we can assume that each coordinate of $\mathbf{x}_1^{(1)}$ is independently equal to $+2$ with probability $\frac{1}{4}$, -2 with probability $\frac{1}{4}$, and 0 with probability $\frac{1}{2}$. Therefore, the probability that the first l_1 coordinates match the expected pattern in $\mathbf{y}_1 + \mathbf{y}_2$ is:

$$\Pr \left[(\mathbf{x}_1^{(1)})_{l_1} = (\mathbf{y}_1 + \mathbf{y}_2)_{l_1} \right] \approx \left(\frac{1}{4} \right)^{l_1/4} \cdot \left(\frac{1}{4} \right)^{l_1/4} \cdot \left(\frac{1}{2} \right)^{l_1/2} = \left(\frac{1}{2} \right)^{3l_1/2}.$$

Note that all the probabilities can be computed exactly for given $\{y_i\}$. We will use this “average-case” behaviour for a fixed Y to estimate the success probability at each level of the algorithm. We emphasise that the following calculations are for this particular choice of Y .

General Setting. For the general case, we model the occurrence of each entry $p' = -n + 2i$ in a vector using Equation (3). Accordingly, we expect approximately $\Pr(X_n = p') \cdot l_j$ occurrences of p' among the corresponding l_j coordinates of y on average. Hence, we estimate the probability of a match at level j of the algorithm as follows:

$$\Pr \left[\left(\sum_{i=1}^n x_i^{(j)} \right)_{l_1+\dots+l_j} = \left(\sum_{i=1}^n y_i \right)_{l_1+\dots+l_j} \right] = \prod_{i=0}^n \Pr(X_n = -n + 2i)^{\Pr(X_n = -n+2i) \cdot l_j}. \quad (12)$$

We acknowledge that this is a crude approximation of a complex probabilistic scenario, but our experimental results indicate that it yields surprisingly accurate predictions in practice.

For ease of representation, we write the success probability at each level j in the simplified form $(\Pr_j)^{l_j}$, where $\Pr_j$ is given by:

$$\Pr_j = \prod_{i=0}^n \Pr(X_n = 2i - n)^{\Pr(X_n = 2i-n)} \quad \text{for } n = 2^j. \quad (13)$$

As in the earlier discussion, in the final step we must consider all remaining coordinates of the target vector Y . Therefore, we define $l_q = m - \sum_{j=1}^{q-1} l_j$. Once we have an estimate for the success probability of a collision, we can compute the expected list sizes at each level and thereby estimate the overall complexity of the algorithm.

6.3 Expected List Lengths

The analysis is the same as in Section 4, just using a different probability for the matches. We assume that all intermediate lists have the same expected length as the initial ones that allows us to give an asymptotic analysis. We now define our lists as follows:

We know that at level j , the list $L_i^{(j)}$ contains $x_i^{(j)}$ which have back-pointers to elements x_{2i-1}^{j-1} and $x_{2i}^{(j-1)}$ of the two child lists used to form $L_i^{(j)}$ such that $x_i^{(j)} = x_{2i-1}^{j-1} + x_{2i}^{j-1}$. This sum matches the corresponding partial sum of 2^j target vectors y , restricted to the specific l_j coordinates. In general, for $i = 1, \dots, k/2^j$ at each level $j = 1, \dots, \log k$,

$$L_i^{(j)} = \left\{ \left(x_i^{(j)}, \pi_i^{(j)} \right) \left| \begin{array}{l} (x_{2i-1}^{(j-1)}, -) \in L_{2i-1}^{(j-1)}, \quad (x_{2i}^{(j-1)}, -) \in L_{2i}^{(j-1)}, \\ x_i^{(j)} = x_{2i-1}^{(j-1)} + x_{2i}^{(j-1)}, \quad (x_i^{(j)})_{l_1+\dots+l_j} = \sum_{r=1}^{2^j} (y^{(2^j)(i-1)+r})_{l_1+\dots+l_j} \\ \pi_i^{(j)} = (\pi_{2i-1}^{(j-1)} \parallel \pi_{2i}^{(j-1)}) \end{array} \right. \right\}. \quad (14)$$

Even though we write it as $(x_i^{(j)})_{l_1+\dots+l_j} = \sum_{r=1}^{2^j} (y^{(2^j)(i-1)+r})_{l_1+\dots+l_j}$, as noted earlier in Remark 2, at each level j one only needs to check the condition for the last l_j entries.

The complexity of the algorithm can be analysed as follows. Using the probability estimation from Section 6.2, we have

$$(\text{Pr}_1)^{l_1} = \Pr \left[(\mathbf{x}_1^{(0)} + \mathbf{x}_2^{(0)})_{l_1} = (\mathbf{y}_1 + \mathbf{y}_2)_{l_1} \right] = \left(\frac{1}{2} \right)^{3l_1/2}.$$

Set the expected list length at each level j to be m_j . Then, by linearity of expectation,

$$m_1 = \mathbb{E}[|L_1^{(1)}|] = |L_1^{(0)}| |L_2^{(0)}| \cdot (\text{Pr}_1)^{l_1} = m_0^2 \cdot \left(\frac{1}{2} \right)^{3l_1/2}.$$

Assuming $\mathbb{E}[|L_1^{(1)}|] = m_0$, we get $m_0 = 2^{3l_1/2}$.

At level 2, we consider the next l_2 coordinates of the vectors and filter pairs from $L_1^{(1)} \times L_2^{(1)}$ whose sum matches the partial sum $\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{y}_3 + \mathbf{y}_4$ restricted to l_2 coordinates. Using a similar argument, we get

$$m_2 = \mathbb{E}[|L_1^{(2)}|] = |L_1^{(1)}| |L_2^{(1)}| \cdot (\text{Pr}_2)^{l_2} = m_0^2 \cdot (\text{Pr}_2)^{l_2} \approx m_0.$$

Substituting $m_0 = 2^{3l_1/2}$, we have $l_2 = \frac{-3l_1}{2\log_2(\text{Pr}_2)}$.

In the final level, we require the expected length of the final list $L_1^{(3)}$ to be at least 1.

$$\mathbb{E}[|L_1^{(3)}|] = |L_1^{(2)}| |L_2^{(2)}| \cdot (\text{Pr}_3)^{l_3} = m_0^2 \cdot (\text{Pr}_3)^{l_3} \geq 1.$$

Note that $l_3 = m - l_1 - l_2$, where m is the dimension of the vectors. Substituting the expressions for m_0 and l_2 into this relation allows us to solve for l_1 in terms of m . This determines the number of coordinates that should match at the first level. By recursively applying this process, we obtain the required initial list length m_0 to find a solution, as well as the number of coordinates l_j that should match at each level j .

General Setting. We extend the idea to any number of lists $k = 2^q$. This gives rise to a binary tree of depth $\log_2 k = q$. At each internal node, we assume that the expected list length is equal to the initial list length m_0 , and derive m_0 and the values of l_j in terms of l_1 . More precisely, we obtain

$$m_0 = \left(\frac{1}{\text{Pr}_1} \right)^{l_1} \quad \text{and} \quad l_j = l_1 \cdot \frac{\log_2(\text{Pr}_1)}{\log_2(\text{Pr}_j)} \quad \text{for } j = 2, \dots, q-1.$$

where Pr_j is the approximate match probability defined in Equation 13. One can also write

$$m_0 = 2^{(\frac{3}{2})l_1} \quad \text{and} \quad l_j = \frac{-3l_1}{2\log_2(\text{Pr}_j)} \quad \text{for } j = 2, \dots, q-1,$$

by substituting the value $\text{Pr}_1 = (\frac{1}{2})^{3/2}$.

At the final level q , we set $l_q = m - \sum_{j=1}^{q-1} l_j$, since the output vector must be of full dimension m . To ensure a solution to the k -sum problem is found with constant probability, we require the expected list length at level q to be at least one:

$$\mathbb{E}[|L_1^{(q)}|] = m_0^2 (\text{Pr}_q)^{m - \sum_{j=1}^{q-1} l_j} \geq 1. \tag{15}$$

Substituting the expressions for m_0 and the l_j values into this equation and taking logarithms, we solve for l_1 . The total sum of the l_j for $j = 1, \dots, q-1$ is given by

$$\sum_{j=1}^{q-1} l_j = l_1 + \sum_{j=2}^{q-1} \frac{-3l_1}{2 \log_2(\text{Pr}_j)}.$$

Thus, the remaining dimension for level q becomes,

$$m - \sum_{j=1}^{q-1} l_j = \frac{-3l_1}{\log_2(\text{Pr}_q)},$$

which simplifies to the expression

$$l_1 = \frac{m}{1 - \frac{3}{2} \sum_{j=2}^{q-1} \frac{1}{\log_2(\text{Pr}_j)} - \frac{3}{\log_2(\text{Pr}_q)}}. \quad (16)$$

By substituting the computed value of l_1 , we can determine m_0 and each l_j , thereby identifying the required initial list length and the number of coordinates that must match at each level $j = 1, \dots, q$ for a final solution to exist with constant probability.

Algorithm 2 summarises how the k -list algorithm works to find the set of $\mathbf{x}_i^{(0)}$'s that add up to the target vector Y . We initialise with $\pi_i^{(0)} = \mathbf{x}_i^{(0)}$ and at each level j , $\pi_i^{(j)}$ acts as the back-pointer that keeps sufficient information to trace back to the original inputs at the level 0.

6.4 Running time & Experimental Evidence

The running time and space complexity of this k -tree algorithm depends on the intermediate list sizes, as discussed in Section 6.4. For this analysis, we assume that all the lists have the same size m_0 throughout the internal steps.

This results in an algorithm for solving the new k -sum problem (Definition 6) with time and space complexity $\mathcal{O}(k \cdot m_0 \log m_0)$, where the list size is $\mathcal{O}(m_0)$, with $m_0 = 2^{3l_1/2}$ and l_1 expressed in terms of the dimension m and the matching probabilities, as given in Equation 16.

For the case $k = 8$, we observed that setting $\lceil l_1 \rceil = \lceil 0.343m \rceil$ allows us to find a solution to the 8-sum problem with non-trivial probability. With this choice, all lists have size approximately $m_0 \approx 2^{0.514m}$, and the algorithm runs in time and space complexity $\tilde{\mathcal{O}}(2^{0.514m})$ ignoring the constant and logarithmic factors.

We observe that the complexity improves as the number of lists increases ($k > 2^3$). In other words, if the target Y is a sum of more than 8 random vectors $\mathbf{y}_i \in \{-1, 1\}^m$, we can solve for Y using a list of random vectors $\mathbf{x}_i \in \{-1, 1\}^m$ in less than $2^{m/2}$ time and space. In Table 3, we present the initial list lengths and the corresponding l_j values calculated asymptotically as a function of the vector dimension m for different numbers of lists $k = 2^q$. As shown, the initial list length m_0 decreases as the number of lists increases. Consequently, the new k -sum problem can be solved faster than square-root time for larger values of k .

In Appendix B, we noticed that solving the 4-sum version of the problem using a basic birthday paradox-type argument requires a total time complexity of $\mathcal{O}(2^m)$. In contrast, based on Table 3, the 4-list algorithm we propose can solve the same problem in approximately $\mathcal{O}(2^{0.6m})$ time (ignoring the log factors). Therefore, we conclude that the k -list algorithm offers a substantial improvement over the classical algorithm. For $k > 8$, the fact that the k -list algorithm

Algorithm 2 k -List Algorithm for a given Y .

Input: $k = 2^q$, $L_i^{(0)}$ for $i = 1, \dots, k$, $Y \in \mathbb{Z}_p^m$.

Output: $\mathbf{x}_1^{(0)}, \dots, \mathbf{x}_k^{(0)}$, or $\perp$

```

1: for  $j = 1, \dots, q$  do
2:   Choose  $l_j$  such that  $\sum_{j=1}^q l_j = m$ .
3:   for  $i = 1, \dots, k/2^j$  do
4:      $L_i^{(j)} \leftarrow \emptyset$ 
5:     Sort  $L_{2i-1}^{(j-1)}$  with respect to the first coordinate,  $\mathbf{x}_{2i-1}^{(j-1)}$ 
6:     for each  $(\mathbf{x}_{2i}^{(j-1)}, \pi_{2i}^{(j-1)}) \in L_{2i}^{(j-1)}$  do
7:       Find all  $(\mathbf{x}_{2i-1}^{(j-1)}, \pi_{2i-1}^{(j-1)}) \in L_{2i-1}^{(j-1)}$  such that

```

$$(\mathbf{x}_{2i-1}^{(j-1)} + \mathbf{x}_{2i}^{(j-1)})_{l_1 + \dots + l_j} = \sum_{r=1}^{2^j} (\mathbf{y}^{(2^j)(i-1)+r})_{l_1 + \dots + l_j}.$$

```

8:       for all such pairs do
9:          $\mathbf{x}_i^{(j)} \leftarrow \mathbf{x}_{2i-1}^{(j-1)} + \mathbf{x}_{2i}^{(j-1)}$ 
10:         $\pi_i^{(j)} \leftarrow (\pi_{2i-1}^{(j-1)} \parallel \pi_{2i}^{(j-1)})$ 
11:        Add  $(\mathbf{x}_i^{(j)}, \pi_i^{(j)})$  to  $L_i^{(j)}$ 
12:       end for
13:     end for
14:   end for
15: end for
16: if  $|L_1^{(q)}| \geq 1$  then
17:   return  $\pi_1^{(q)}$ 
18: else
19:   return  $\perp$ 
20: end if

```

achieves time and space complexities below the square-root bound $\mathcal{O}(2^{m/2})$ makes it superior to other known algorithms for high-dimensional problems where traditional approaches become infeasible.

Remark 4. Figure 4 presents experimental evidence to support the assumption that the expected list length m_j remains approximately equal to m_0 across all levels of the algorithm. In this experiment, we considered small vector lengths ($m = 50, 60$) and computed the theoretical expected list lengths m_j at each level of the k -sum problem for $k = 2^9$ and $k = 2^7$. These theoretical estimates were then compared with the experimental results. The experiments were conducted using small parameter sets to ensure computational feasibility.

For the rounded l_j values computed at each level, the theoretically expected list lengths were computed by the relation $m_j = m_0^2 (\text{Pr}_j)^{l_j}$ and written in terms of m_0 at each level. As shown in the results, the theoretical expectation $m_j = m_0$ does not hold exactly due to the rounding errors in the l_j values. Table 4 lists the specific l_j values used in the experiments, along with the corresponding theoretical estimates of m_j .

k	l_1	l_2	l_3	l_4	l_5	l_6	l_7	$\log_2(m_0)$
4	0.4037m	0.5963m						0.6055m
8	0.3427m	0.2532m	0.4041m					0.5141m
16	0.3018m	0.2230m	0.1780m	0.2972m				0.4528m
32	0.2727m	0.2015m	0.1608m	0.1343m	0.2307m			0.4091m
64	0.2510m	0.1854m	0.1480m	0.1236m	0.1061m	0.1860m		0.3764m
128	0.2340m	0.1728m	0.1379m	0.1152m	0.0989m	0.0867m	0.1544m	0.3510m

Table 3: Examples for the initial list lengths and l_j values for $j = 1, \dots, q$, for various numbers of lists k .

$$m = 50, m_0 = 2^{18}, k = 2^7$$

Rounded l_j	12	9	7	6	5	4	7
Expected m_j	$1.0m_0$	$0.8m_0$	$1.1m_0$	$0.8m_0$	$1.2m_0$	$3.5m_0$	18

$$m = 50, m_0 = 2^{17}, k = 2^9$$

Rounded l_j	11	8	6	5	5	4	4	3	4
Expected m_j	$1.4m_0$	$1.7m_0$	$3.3m_0$	$3.4m_0$	$0.6m_0$	$1.8m_0$	$0.4m_0$	$3.6m_0$	3594

$$m = 60, m_0 = 2^{20}, k = 2^9$$

Rounded l_j	13	10	8	6	5	5	4	4	5
Expected m_j	$1.4m_0$	$0.8m_0$	$0.8m_0$	$3.3m_0$	$4.8m_0$	$0.8m_0$	$3.5m_0$	$0.9m_0$	4920

Table 4: The rounded l_j and the Expected list lengths, m_j for $j = 1, \dots, \log_2(k)$. This data is used in the experiment for Figure 4.

To compare the theoretical results with experimental findings, we test for merges in isolation for each level of the k -list algorithm for the same values of $k = 2^7, 2^9$ and vector dimensions $m = 50, 60$. For this experiment, we fixed the parameter l_j according to Table 4, and considered vectors of length l_j , since this is the number of coordinates matched at a fixed level j .

Over 100 trials, we recorded the range of values attained by the experimental list lengths $|L^{(j)}|$ at different levels of the k -tree algorithm, in order to evaluate their deviation from the theoretical estimates m_j provided in Table 4. As shown in Figure 4, the experimental results demonstrate that, with high probability, the list lengths fall within the range $[m_j/4, 4m_j]$ at each level. In this figure, the x -axis corresponds to the levels of the k -tree algorithm, while the y -axis shows the frequency of list lengths falling into the respective intervals.

Across all nodes in the tree, some lists may be slightly larger than the expected value, while others may be slightly smaller. However, our experiments show that these deviations are not significant. In practice, the fluctuations tend to balance out when the lists are merged: above-average-sized lists compensate for those that are below average, maintaining overall stability in the merging process. This experimental and theoretical comparison supports and validates our assumption that $m_j \approx m_0$ across levels.

It is also important to note that the expected list length m_j depends on the success probability calculations at each level. Therefore, the fact that the experimental outcomes closely match

the theoretical expectations provides indirect validation of the accuracy of our average-case success probability estimates as well.

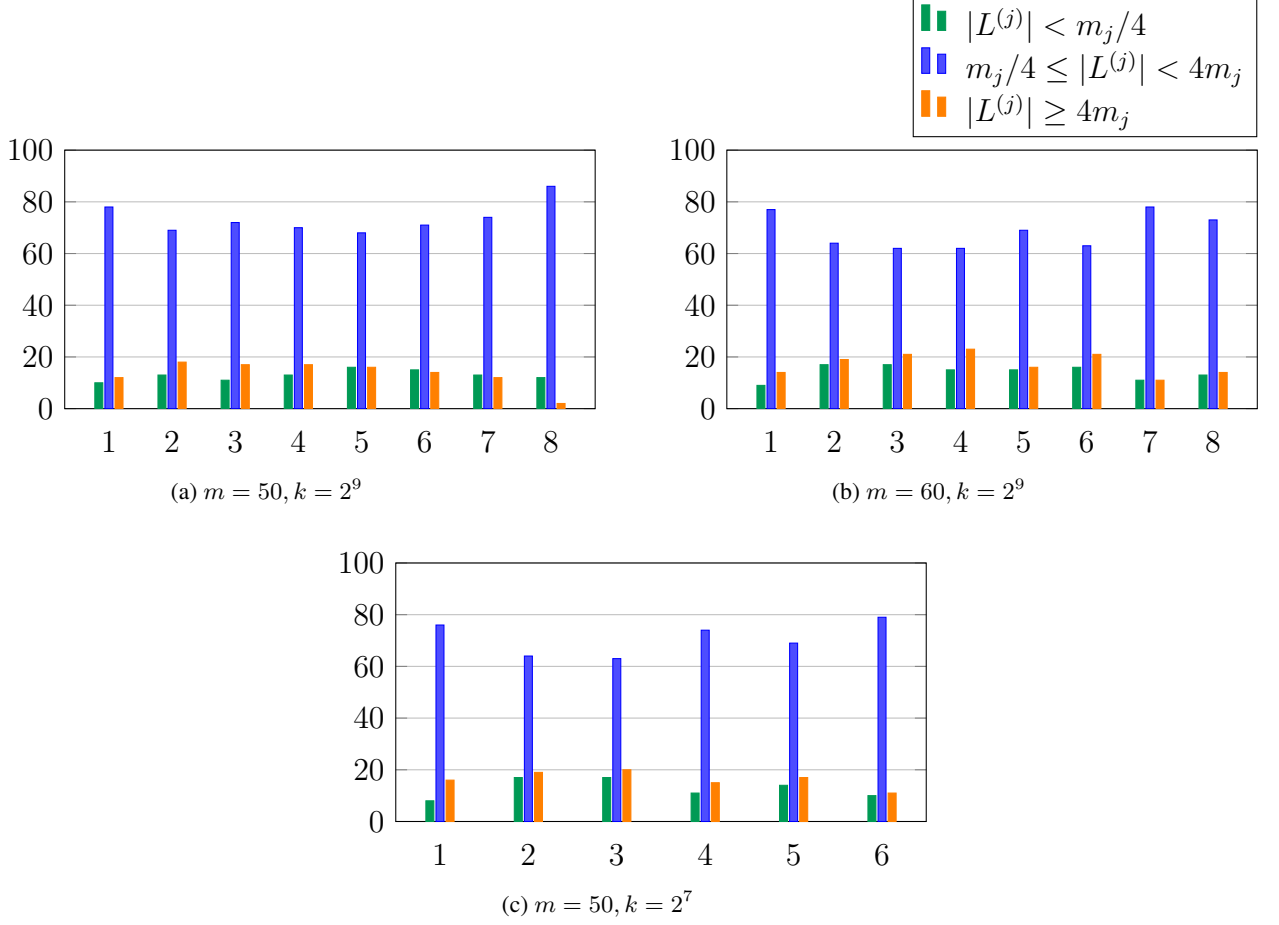


Fig. 4: Experimental results for $|L^{(j)}|$ for different m and k

7 Average Collision Probability & Overall Running Time

All computations and experiments in Section 6.4 were carried out for a particular choice of fixed Y , as introduced in Section 6.2. These results illustrate that the running time of the algorithm depends inherently on the chosen target Y and may vary significantly between instances. In extreme worst-case scenarios, for example, when $Y = (\pm k, \dots, \pm k)$, the cost of finding a match can be exceptionally large; however, such extreme instances occur with negligible probability under the natural distribution of Y . At the opposite extreme, when Y is the zero vector, the problem essentially reduces to the zero-sum case discussed in Section 3. This naturally raises the question whether we can define and compute an *overall running time estimate* by averaging over many possible fixed Y 's.

To formalise this idea, let $L = \{X_1, \dots, X_N\}$ be a list of independent random vectors sampled from a distribution $\mathcal{D}$. Here $\mathcal{D}$ is the distribution of the sum of k independently and uniformly chosen vectors from $\{-1, 1\}^m$, that is, $X_i = \sum_{l=1}^k \mathbf{x}_{i,l}$, where $\mathbf{x}_{i,l} \leftarrow \{-1, 1\}^m$.

Similarly, for each target we generate $Y_j = \sum_{l=1}^k \mathbf{y}_{j,l}$, where $\mathbf{y}_{j,l} \leftarrow \$ \{-1, 1\}^m$, so that each Y_j is an independent sample from the same distribution $\mathcal{D}$. Assume throughout that $X_1, \dots, X_N$ and $Y_1, \dots, Y_M$ are independent and identically distributed (i.i.d.) samples from $\mathcal{D}$ and that the two collections are independent.

For a given target Y_1 , the probability that a randomly chosen $X \in L$ equals Y_1 can be expressed as

$$\Pr(X = Y_1) = \frac{\text{total number of matches in } L}{N}.$$

Equivalently,

$$\text{total number of matches in } L = \sum_{i=1}^N \mathbf{1}_{\{X_i=Y_1\}},$$

where $\mathbf{1}_{\{X_i=Y_1\}} \in \{0, 1\}$ is the indicator of the event $X_i = Y_1$.

Repeating this experiment for M independently sampled targets $Y_1, \dots, Y_M$, the *average matching probability* is

$$\frac{\text{total number of matches over all targets}}{NM} = \frac{1}{NM} \sum_{j=1}^M \sum_{i=1}^N \mathbf{1}_{\{X_i=Y_j\}}.$$

This can be rewritten as

$$\frac{1}{M} \sum_{j=1}^M \left(\frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{X_i=Y_j\}} \right).$$

For each fixed Y_j , the inner average $\frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{X_i=Y_j\}}$ is the empirical probability that Y_j matches an element of the list L . By the law of large numbers, as N grows, this empirical probability converges to the probability $\Pr(X = Y_j)$ that a random $X \leftarrow \$ \mathcal{D}$ equals a fixed target Y_j . Consequently,

$$\frac{1}{NM} \sum_{j=1}^M \sum_{i=1}^N \mathbf{1}_{\{X_i=Y_j\}} \approx \frac{1}{M} \sum_{j=1}^M \Pr(X = Y_j).$$

Applying the law of large numbers again, now for large M , this approximates to the probability that X equals a random $Y \leftarrow \$ \mathcal{D}$.

$$\frac{1}{M} \sum_{j=1}^M \Pr(X = Y_j) \approx \Pr(X = Y),$$

where the last probability is precisely the *collision probability* between two independent random variables from the distribution $\mathcal{D}$. Thus, when both N and M are large, estimating the average matching probability over many fixed targets is asymptotically equal to computing the collision probability, i.e., the probability that two independent samples $X, Y \leftarrow \$ \mathcal{D}$ are equal.

Since both X and Y are sums of k random $\{-1, 1\}^m$ vectors, the coordinates can take any value from $\{-k, -k+2, \dots, k\}$. In a k -steps random walk, the probability that a single coordinate equals $p' \in \{-k, -k+2, \dots, k\}$ is

$$\Pr(x = p') = 2^{-k} \binom{k}{\frac{k+p'}{2}}.$$

The standard collision probability calculations rely on the randomness and independence of both summands, and compute the collision probability as the sum of squared probabilities of all possible outcomes in a k -step random walk. Consequently, the probability that two vectors of dimension m sampled uniformly at random from $\mathcal{D}$ have a match is given by,

$$\Pr(X = Y) = \left(\sum_{p'} \Pr(x = p')^2 \right)^m. \quad (17)$$

This compact formula provides a direct theoretical estimate for the average-case matching probability, and hence one can use it to compute the overall average running time of the algorithm. To verify the validity of our claim, we conduct the following experiment.

7.1 Experimental Evidence

This experiment estimates the average probability of collision between a randomly chosen target vector Y_j and a list of vectors X_i where both Y_j and each X_i are generated as the sum of k independent random vectors sampled uniformly from $\{-1, 1\}^m$. We compare the empirical collision probabilities with the theoretical estimates, and also evaluate both the actual and expected number of collisions.

Experiment Procedure:

1. Generate a list $L = \{X_1, X_2, \dots, X_N\}$, where each vector X_i is constructed as $X_i = \sum_{l=1}^k x_{i,l}$ with $x_{i,l} \leftarrow \{-1, 1\}^m$.
2. Fix a target vector (say Y_1) as $Y_1 = \sum_{l=1}^k y_{1,l}$ with $y_{1,l} \leftarrow \{-1, 1\}^m$.
3. For a fixed target Y_1 , count how many vectors in L exactly match Y_1 . The collision probability for a fixed Y_1 is given by:

$$\Pr(X = Y_1) = \frac{\text{Total number of matches}}{N}.$$

4. Experimental average:

Repeat steps 1–3 for distinct target vectors Y_j , $j = 1, \dots, M$, and compute the average probability of collision over all M targets:

$$\text{Avg. } \Pr(X = Y) = \frac{1}{M} \sum_{j=1}^M \Pr(X = Y_j). \quad (18)$$

The corresponding average number of matches per target Y_j is:

$$\text{Avg. number of matches} = \frac{\sum_{j=1}^M (\text{Total number of matches per } Y_j)}{M}.$$

5. Theoretical estimates:

Assuming each X and Y are independent and randomly chosen samples from the same distribution, the theoretical collision probability $\Pr(X = Y)$ is calculated using Equation (17). The expected number of matches for a fixed Y is:

$$\mathbb{E}[\text{Theoretical Matches}] = N \cdot \Pr(X = Y).$$

6. Finally, compare the collision probabilities using theoretical estimates and empirical results given by Equation (17) and Equation (18) respectively. Moreover, the empirical average number of matches (from Step 4) were compared with the theoretical expected number of matches (from Step 5).

We perform this experiment for various parameter settings. The results, shown in Table 5, demonstrate that the empirical averages for both collision probability and number of matches closely align with the theoretical predictions. This agreement supports the conclusion that, for average running time analysis, standard collision probability estimates are reliable.

List Size (N)	Fixed Y_j (M)	Sum (k)	Dim. (m)	Avg Collision Pr (Experiment)	Collision Pr (Theory)	Avg Matches (Experiment)	Exp Matches (Theory)
300	300	10	5	0.000167	0.000170	0.050222	0.050947
300	300	2	4	0.019666	0.019775	5.899778	5.932617
500	300	8	8	0.000058	0.000057	0.029091	0.028679
500	300	16	5	0.000055	0.000054	0.027273	0.026843
300	500	6	6	0.000121	0.000132	0.036182	0.039536
300	500	16	5	0.000053	0.000054	0.015818	0.016106

Table 5: Comparison of empirical and theoretical collision probabilities and expected matches.

7.2 Running Time Estimate

We now use the collision probabilities to derive a heuristic estimate of the algorithm's running time. Consider the basic meet-in-the-middle algorithm for $k = 4$. Let $X = \sum_{i=1}^4 \mathbf{x}_i$, and $Y = \sum_{i=1}^4 \mathbf{y}_i$, where X and Y are independent random variables drawn from the same distribution. Using the standard collision probability method, and from the properties of a random walk, the possible coordinate values for each vector are $\{-4, -2, 0, 2, 4\}$. In other words we have taken $k = 2^2$ steps in the random walk. Therefore, the collision probability for a single coordinate is

$$P_Y = \left(\frac{1}{2^4}\right)^2 + \left(\frac{4}{2^4}\right)^2 + \left(\frac{6}{2^4}\right)^2 + \left(\frac{4}{2^4}\right)^2 + \left(\frac{1}{2^4}\right)^2 = 0.27344.$$

For an m -dimensional vector, the probability of a coordinate-wise match is then $P_Y^m = (0.27344)^m$.

Following the discussion on Appendix B, assume you have two lists of size N . The probability that at least one match happens is

$$\Pr(\text{a match happens}) = 1 - (1 - P_Y^m)^{N^2},$$

and to ensure a success probability of at least $1/2$, we require $N \geq \sqrt{\ln 2} \cdot P_Y^{-m/2}$. Since $P_Y = 0.27344$, this gives $N \approx (2^{0.93535m})$. The resulting total complexity of the algorithm is, therefore $\mathcal{O}(2^{0.93535m})$. Compared to the fixed-target case from Appendix B, where $Y = (\sqrt{k}, \dots, \sqrt{k})$ led to a running time of $\mathcal{O}(2^m)$, the average-case running time is slightly better, but still significantly above square-root complexity.

Next, we show how these calculations can be used to determine the overall running time estimate of the k -tree algorithm over multiple target vectors Y_j . Recall that the k -tree algorithm

proceeds through q levels, where $k = 2^q$. For a fixed target Y , the collision probability at each level j was computed in Section 6.2 using Equation (13). We can now replace this with the average collision probability, given by

$$\text{Average collision probability at level } j = \Pr_j(X = Y) = \sum_{p'} \Pr(x_n = p')^2, \quad (19)$$

where $n = 2^j$ is the number of steps in the random walk and $p' \in \{-n, -n+2, \dots, n\}$. Table 6 compares the two collision probabilities across the different levels j of the algorithm.

Level j	Collision Probability for a fixed Y Equation (13)	Avg. Collision Probability Equation (19)
1	0.35355	0.375
2	0.24475	0.27344
3	0.17144	0.19638
4	0.12103	0.13995
5	0.08556	0.09935
6	0.06049	0.07039
7	0.04278	0.04982

Table 6: Comparison of fixed and average collision probabilities for each level j .

Once the collision probability at each level is determined, the subsequent calculations proceed exactly as in the fixed- Y case. An asymptotic analysis can be carried out by taking the list sizes to be equal at each stage and determining the matching coordinates l_j as described in Section 6.4. Table 7 illustrates how the change in collision probabilities affect the initial list length m_0 , which determines the running time of the algorithm when it succeeds in finding a solution. The impact is more evident when the number of lists k is small than when it is large. Note that when $k = 4$, the k -tree algorithm achieves a better approximate running time of $\mathcal{O}(2^{0.56313m})$ compared to $\mathcal{O}(2^{0.93535m})$ for the basic meet-in-the-middle algorithm. The k -tree algorithm shows improved running time estimate when moving from the fixed-target case to the average case. While this provides insight into the algorithm's average performance when it always solves the instance, we emphasise that its actual running time always depends on the specific target vector Y .

k	4	8	16	32	64	128
$\log_2(m_0)$ for a fixed Y	0.6055 m	0.5141 m	0.4528 m	0.4091 m	0.3764 m	0.3510 m
$\log_2(m_0)$ for average Y	0.56313 m	0.47780 m	0.42157 m	0.39340 m	0.35674 m	0.33287 m

Table 7: Comparison of the initial list lengths

Remark 5. Note that the collision probability computed via Equation (17) coincides with the success probability derived in Section 3 for the zero-sum version of the k -tree algorithm, where

k is effectively replaced by $2k$. In other words, for average running time calculations, we can consider

$$\Pr \left(\sum_{i=1}^k \mathbf{x}_i = \sum_{i=1}^k \mathbf{y}_i \right) = \Pr \left(\sum_{i=1}^{2k} \mathbf{z}_i = \mathbf{0} \right),$$

where $\mathbf{x}_i, \mathbf{y}_i, \mathbf{z}_i \leftarrow \{-1, 1\}^m$ are independent random vectors.

8 Conclusion and future work

This paper focused on algorithmic problems related to the generalised birthday problem. We first introduced a new variant of this problem over $\{-1, 1\}^m$ and showed that the new version is harder than the original generalised birthday problem with $\{0, 1\}$ vectors. We proposed Wagner-style k -tree algorithms that outperform naive meet-in-the-middle approaches, providing both theoretical analyses and experimental estimates. We then applied this algorithmic framework to the recently proposed randomised one-more Inhomogeneous Short Integer Solution (rOM-ISIS) assumption, showing that additional solutions can be generated efficiently. While this does not currently compromise the associated blind signature scheme under published parameters, it demonstrates that the assumption itself is weaker than intended. Finally, we introduced and analysed a generalised variant of the k -sum problem with a fixed target vector, presenting the first systematic treatment of this problem and providing both heuristic analyses and suggestions for empirical validation.

Overall, our work highlights the importance of analysing structured variants of the generalised birthday problem and their implications for cryptographic security. As a future project, we suggest implementation and empirical validation of the proposed k -tree algorithms would provide practical insights, confirm theoretical predictions, and guide refinements beyond heuristic analysis. We invite further exploration of these techniques and their impact on the design and analysis of cryptographic schemes.

References

1. Shweta Agrawal, Elena Kirshanova, Damien Stehlé, and Anshu Yadav. Practical, round-optimal lattice-based blind signatures. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 39–53, 2022.
2. Miklós Ajtai. Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.
3. Shi Bai, Steven D Galbraith, Liangze Li, and Daniel Sheffield. Improved combinatorial algorithms for the Inhomogeneous Short Integer Solution problem. *Journal of Cryptology*, 32:35–83, 2019.
4. Foteini Baldimtsi, Jiaqi Cheng, Rishab Goyal, and Aayush Yadav. Non-interactive blind signatures from lattices. In *Asiacrypt 2024*. Springer, 2024.
5. Foteini Baldimtsi, Rishab Goyal, and Aayush Yadav. Batched & non-interactive blind signatures from lattices. *Cryptology ePrint Archive*, 2025.
6. Dan Boneh and David Mandell Freeman. Linearly homomorphic signatures over binary fields and new tools for lattice-based signatures. In *International Workshop on Public Key Cryptography*, pages 1–16. Springer, 2011.
7. Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Alessandro Sorniotti. A framework for practical anonymous credentials from lattices. In *Crypto 2023*, pages 384–417. Springer, 2023.
8. Leif Both and Alexander May. The approximate k-list problem. *IACR Transactions on Symmetric Cryptology*, pages 380–397, 2017.
9. Leif Both and Alexander May. Decoding linear codes with high error rate and its impact for LPN security. In *PQCrypto 2018*, pages 25–46. Springer, 2018.
10. Paul Camion and Jacques Patarin. The knapsack hash function proposed at crypto’89 can be broken. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 39–53. Springer, 1991.
11. Itai Dinur. An algorithmic framework for the Generalized birthday problem. *Designs, Codes and Cryptography*, 87:1897–1926, 2019.

12. Sebastian Faller, Guilhem Niot, and Michael Reichle. Lattice-based threshold blind signatures. *Cryptology ePrint Archive*, 2025.
13. Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-Fourier lattice-based compact signatures over NTRU. *Submission to the NIST's post-quantum cryptography standardization process*, 36(5):1–75, 2018.
14. Steven D Galbraith. *Mathematics of public key cryptography*. Cambridge University Press, 2012.
15. Lucjan Hanzlik. Non-interactive blind signatures for random messages. In *Eurocrypt 2023*, pages 722–752. Springer, 2023.
16. Ellis Horowitz and Sartaj Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM (JACM)*, 21(2):277–292, 1974.
17. Antoine Joux, Hunter Kippen, and Julian Loss. A concrete analysis of Wagner's k-list algorithm over $\mathbb{Z}_p$. *Cryptology ePrint Archive*, 2024.
18. Paul Kirchner. Improved Generalized birthday attack. *Cryptology ePrint Archive*, 2011.
19. Vadim Lyubashevsky. On random high density subset sums. *Electronic Colloquium on Computational Complexity*, TR05, 2005.
20. Vadim Lyubashevsky. The parity problem in the presence of noise, decoding random linear codes, and the subset sum problem. In *International Workshop on Approximation Algorithms for Combinatorial Optimization*, pages 378–389. Springer, 2005.
21. Yannis Manolopoulos. Binomial coefficient computation: recursion or iteration? *ACM SIGCSE Bulletin*, 34(4):65–67, 2002.
22. Alexander May and Ilya Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In *Eurocrypt 2015*, pages 203–228. Springer, 2015.
23. Daniele Micciancio. Generalized compact knapsacks, cyclic lattices, and efficient one-way functions. *computational complexity*, 16:365–411, 2007.
24. Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on Gaussian measures. *SIAM journal on computing*, 37(1):267–302, 2007.
25. Lorenz Minder and Alistair Sinclair. The extended k-tree algorithm. *Journal of cryptology*, 25:349–382, 2012.
26. Ivica Nikolić and Yu Sasaki. Refinements of the k-tree algorithm for the Generalized birthday problem. In *Asiacrypt 2015*, pages 683–703. Springer, 2015.
27. Richard Schroepel and Adi Shamir. A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems. *SIAM journal on Computing*, 10(3):456–464, 1981.
28. B. I. Selivanov. On waiting time in the scheme of random allocation of coloured particles. *Discrete Mathematics and Applications*, 5(1):73–82, 1995.
29. Andrew Shallue. An improved multi-set algorithm for the dense subset sum problem. In *International Algorithmic Number Theory Symposium*, pages 416–429. Springer, 2008.
30. David Wagner. A Generalized birthday problem. In *CRYPTO*, pages 288–304. Springer, 2002.

A Naive birthday algorithms

To give intuition about how our problem is different from Wagner's original binary xor problem, we discuss naive (pre-Wagner) ways to solve the 4-list problem using the birthday paradox.

Suppose one is given 4 lists L_1, L_2, L_3, L_4 of binary strings in $\{0, 1\}^m$ of the same size, and wants to find $x_i \in L_i$ such that $x_1 \oplus x_2 \oplus x_3 \oplus x_4 = 0$. One can expect a solution as soon as $|L_i| = 2^{m/4+o(1)}$. If one had never heard of Wagner's algorithm, but knew about baby-step-giant-step, then the following algorithm would be natural: Compute $L_{12} = \{(x_1 \oplus x_2, x_1, x_2) : x_1 \in L_1, x_2 \in L_2\}$ and sort it with respect to the first coordinate. Then we expect $|L_{12}| > 2^{m/2}$. Similarly, let $L_{34} = \{(x_3 \oplus x_4, x_3, x_4) : x_3 \in L_3, x_4 \in L_4\}$. For each $v \in L_{34}$ search for v in the list L_{12} . By the birthday paradox (the “two lists version”) one expects a match if $|L_{12}|, |L_{34}| > 2^{m/2}$. In other words, there is a $\tilde{O}(2^{m/2})$ algorithm.

Now let us consider our problem. Suppose L_1, L_2, L_3, L_4 are lists of vectors in $\{-1, 1\}^m$ of the same size. We seek $x_i \in L_i$ such that $x_1 + x_2 + x_3 + x_4 = 0$.

Let $L_{12} = \{(x_1 + x_2, x_1, x_2) : x_1 \in L_1, x_2 \in L_2\}$ and $L_{34} = \{(x_3 + x_4, x_3, x_4) : x_3 \in L_3, x_4 \in L_4\}$. These are lists of vectors with entries in $\{-2, 0, 2\}$. The probability that an entry is 0 is $p_0 = 1/2$, and the probability of +2 and -2 both $p_{-2} = p_2 = 1/4$. Sort L_{12} with respect to the first coordinate. What is the probability that there exists $v \in L_{34}$ such that $v \in L_{12}$?

Fortunately, this non-uniform birthday problem has been solved. A good reference is the paper by Selivanov.

Let p_v be the probability of a vector $v \in \{-2, 0, 2\}^m$ being generated in a list. We get p_i by just applying the above probability distribution to each entry. Selivanov defines $v_2 = \sum_v p_v^2$. In our case, one can see that

$$v_2 = (p_{-2}^2 + p_0^2 + p_2^2)^m = 0.375^m.$$

Selivanov treats the problem as a “coloured” problem. The vectors in L_{12} are one colour and the vectors in L_{34} are the other colour. Colours are chosen independently with probability $1/2$. The expected number of trials until the same vector v is chosen but coloured in both ways is

$$\sqrt{\pi/v_2}(1 + o(1)).$$

This evaluates in our example to $\sqrt{\pi}2.66^{m/2} = \sqrt{\pi}2^{0.7m}$.

This immediately implies that we expect to get a match when $|L_{12}|, |L_{34}| \approx (\sqrt{\pi}/2)2^{0.7m}$. From this we deduce that the original lists L_i should have size $2^{0.35m}$ and the algorithm complexity is $\tilde{O}(2^{0.7m})$. One sees this is a substantially harder problem than the binary vector case with respect to this simple class of algorithms.

Of course, we know about Wagner’s algorithm, so the results in this appendix don’t tell anything about the best algorithm for the problem. But hopefully they help the reader understand the nature of the birthday paradox in the setting of our problem.

B Birthday paradox for a Given vector

Before designing an efficient algorithm for the new k -sum problem 6, it is important to understand the inherent hardness of the problem and the type of complexity one might expect. In this variant, the goal is to find a sum of random ± 1 vectors that equals a fixed vector $Y \in \mathbb{Z}^m$. The fixed nature of Y significantly complicates the problem, since the probability that a sum of random vectors (which approximately follows a normal distribution) matches a specific target depends on this target Y . As a result, standard collision-based complexity estimates such as those in [28] are not directly applicable.

In general, the collision-based techniques assume that both sides of the equality are independent random variables drawn from the same distribution. However, in our setting, the target vector $Y = \sum_{i=1}^k \mathbf{y}_i$ is fixed and determined by known input vectors $\mathbf{y}_i \in \{-1, 1\}^m$. The task is to find vectors $\mathbf{x}_i \in L_i$, where each L_i contains independently and uniformly chosen random vectors, such that $\sum_{i=1}^k \mathbf{x}_i = Y$. Since Y is not a random variable, it is incorrect to model this as a symmetric collision event between two random sums. Therefore, the standard technique of computing the collision probability as the sum of squared probabilities does not apply for a given instance.

To further illustrate this, consider a basic meet-in-the-middle approach. Suppose the target vector Y is the sum of $k = 4$ random vectors $\mathbf{y}_i \in \{-1, 1\}^m$, and we are given four lists $L_1, \dots, L_4$ of random vectors. The task is to find vectors $\mathbf{x}_i \in L_i$ such that $\mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3 + \mathbf{x}_4 = Y$. We define two intermediate sum lists,

$$L_{12} = \{\mathbf{x}_1 + \mathbf{x}_2 : \mathbf{x}_1 \in L_1, \mathbf{x}_2 \in L_2\} \quad \text{and} \quad L_{34} = \{Y - (\mathbf{x}_3 + \mathbf{x}_4) : \mathbf{x}_3 \in L_3, \mathbf{x}_4 \in L_4\}.$$

A solution is found when an element from L_{12} matches an element from L_{34} . However, when Y is fixed, the distributions of the vectors in L_{12} and L_{34} are no longer identical. In particular, L_{12} consists of sums of independent random vectors, while L_{34} is offset by the fixed target Y , which introduces dependence. As a result, these two lists are not symmetric random variables, and the standard assumptions behind collision probability estimates no longer hold.

Instead, we employ a heuristic based on the birthday paradox to obtain a lower bound on the complexity of the problem. The key idea is that the sum of random ± 1 vectors $Y = \sum_{i=1}^k \mathbf{y}_i$ in each coordinate behaves like a one-dimensional random walk, which can be approximated by a normal distribution. After k steps, each coordinate has mean 0 and variance k , so the typical size of a coordinate is on the order of $\sqrt{k}$. As a simplifying heuristic, we therefore approximate $Y \approx (\sqrt{k}, \dots, \sqrt{k})$. While this assumption oversimplifies the true distribution of Y , we believe it suffices for a rough complexity estimate.

Let P_Y denote the probability that a sum of $k = 4$ random ± 1 values in a single coordinate equals $\sqrt{k} = 2$. This follows a binomial distribution:

$$P_Y = 2^{-k} \binom{k}{\frac{k+\sqrt{k}}{2}} = 2^{-4} \binom{4}{3} = 0.25.$$

Since $Y \in \mathbb{Z}^m$, the probability of matching all m coordinates simultaneously is $P_Y^m = (0.25)^m$. Now, suppose you have two lists each of size N . Then the probability that at least one collision occurs is:

$$\Pr(\text{a match occurs}) = 1 - (1 - P_Y^m)^{N^2}.$$

Applying the standard approximation $(1 - x)^a \approx e^{-ax}$ for small x , we get

$$\Pr(\text{a match occurs}) \approx 1 - e^{-N^2 \cdot P_Y^m}.$$

To ensure a success probability of at least $1/2$, we require:

$$e^{-N^2 \cdot P_Y^m} < \frac{1}{2} \implies N > \sqrt{\ln 2} \cdot P_Y^{-m/2}.$$

Since $P_Y = 0.25 = 2^{-2}$, this implies $N > \sqrt{\ln 2} \cdot (2^m)$, and hence, the overall time and space complexity of the algorithm is $\mathcal{O}(2^m)$.

As the number of summands increases, the complexity of this approach grows accordingly. For example, when $k = 16$ and $Y = (4, \dots, 4)$, we have $P_Y = 2^{-16} \binom{16}{10} = 0.1222$. This gives $N \approx 2.86^m$, and the algorithm runs in time $\mathcal{O}(2^{1.516m})$. If we instead consider the mean value, which is zero, then Y is the zero vector and the sum of four vectors yields $P_Y = 2^{-4} \binom{4}{2} = 0.375$. In this case $N \approx 2^{0.7075m}$. Thus, for each instance, the running time depends on the chosen target Y .